The Australian National University

School of Computing

JHQ-GPU: Representation-Aware GPU ANN Search and Workload-Level Refinement Calibration

Rui Luo

Supervisor: Dr Mengxuan Zhang

A research thesis submitted for

COMP8800 — Advanced Computing Research Project

October 2026

DRAFT TEMPLATE — WRITING IN PROGRESS

Declaration and Contributions

TO WRITE: Replace this page with the declaration required by the current Canvas instructions. State the work you personally performed, identify inherited JHQ algorithms and code, and distinguish your contributions from those of the supervisor, collaborators, and generative-AI tools. Do not sign an unverified claim of sole authorship or prior non-submission.

Research ownership

TO WRITE: Describe your contribution to system design, implementation, experimental design, analysis, and writing. Cite the original JHQ paper and identify the relationship to the coauthored ADC manuscript.

Use of generative-AI tools

TO WRITE: Complete the record in Appendix C. Record actual tools, purposes, relevant prompts or prompt-log locations, and your review of outputs. Match the current course disclosure instructions.

Signature and date

TO WRITE: Add a signature and date only if the course requires them and after checking the final declaration wording.

Abstract

TO WRITE: Write this after the results and conclusion. A useful planning length is 250–350 words, subject to Canvas instructions. Cover the problem, your own methods, the strongest verified results, and the principal quality or deployment limitation. Keep experimental versions consistent. Do not describe this template or the writing blueprint in the final abstract.

Acknowledgements

TO WRITE: Acknowledge actual supervision, collaboration, computing resources, and other assistance. This page is optional unless the course specifies it; remove its input from main.tex if unused.

Contents

Declaration and Contributions i

Abstract ii

Acknowledgements iii

Notation and Abbreviations xii

1 Introduction 1

1.1 Context and Practical Objectives 1
1.2 Problem Definition and Scope 1
1.3 Three Linked Technical Challenges 1
1.4 Research Questions and Falsifiable Claims 1
1.5 Contributions and Research Ownership 1
1.6 Thesis Roadmap 2

2 Background and Critical Related Work 3

2.1 Formal ANN Objective and Metrics 3
2.2 IVF, PQ, ADC and Residual Refinement 3
2.3 Inherited JHQ: Rotation, Cartesian Codes and Assumptions 3
2.4 GPU Execution Concepts Used Here 3
2.5 GPU Graph Search: SONG, CAGRA and BANG 4
2.6 Quantised ANN, Fast Tables and Modern Alternatives 4
2.7 Critical Synthesis and Precise Research Gap 4

3 Research Design and Measurement Methodology 5

3.1 Research Questions as Controlled Comparisons 5
3.2 Datasets, Queries, Ground Truth and Splits 5
3.3 Measurement Boundary, Variance and Memory Accounting 5
3.4 Baselines, Tuning Grids and Frontier Construction 5
3.5 Versioned Claim-to-Evidence Ledger 6
3.6 Evidence Strength and Threat Controls 6

4 System Architecture and GPU Index Construction **7**
4.1 End-to-End Build and Search Boundaries 7
4.2 Training, Rotation and Codebook Generation 7
4.3 IVF Assignment, IDs and Physical Memory Layout 7
4.4 Resident Storage, Workspace and Index Lifecycle 7
4.5 Construction Cost and Tested Scale 8
4.6 Index Representation and Safety Contracts 8

5 Representation-Aware GPU Search Execution **9**
5.1 Query Path and Four Distinct Ranking Objects 9
5.2 Exact Cartesian LUT Factorisation 11
5.3 Packed Codes and Candidate-Parallel Memory Access 13
5.4 Exact Global Primary Top-c via Multi-Block Scanning 15
5.5 Shared Complete-Key Thresholds and Execution Scheduling 18
5.6 Lane-Cooperative Residual Scoring and Final Top-k 22
5.7 IVF Probe Selection as an Independent Primitive 25

6 Workload-Level Refinement-Budget Calibration **28**
6.1 Why Fixed Refinement Budgets Can Be Wasteful 28
6.2 GT-Free Sampled Output Agreement 28
6.3 Budget Search, Nestedness and Tie Safety 28
6.4 Budget Baselines and Their Questions 28
6.5 Hold-out Risk, Calibration Cost and Payback 29
6.6 Policy Failure Modes and Applicability 29

7 Experimental Evaluation and Research Findings **30**
7.1 Final Experiment Manifest and Comparison Families 30
7.2 Modern GPU ANN Recall–QPS Frontiers 32
7.3 Cartesian LUT, Packed Loads and Lane Refinement 36
7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation 39
7.5 Held-out Budget Quality, Throughput and Payback 42
7.6 Batch Regimes, G Scheduling and Workload Skew 47
7.7 Cold Construction, Resident Memory and Tested Scale 50
7.8 Numerical Robustness, Anomalies and Recall Attribution 54

8 Discussion, Limitations and Future Research **58**
8.1 Three Cross-Layer Insights 58

8.2 Two Deep Negative-Result Case Studies 58
8.3 Threats to Internal and External Validity 58
8.4 Original Testable Research Directions 58
8.5 Conditional Deployment Recommendation 58

9 Conclusion **60**
9.1 Answers to the Four Research Questions 60
9.2 Closing Principle and Outlook 60

Bibliography **61**

A Reproducibility and Experiment Manifest **63**
A.1 Source and Build Environment 63
A.2 Data and Trained States 64
A.3 Result-to-Figure Mapping 65

B Supplementary Results and Correctness Checks **67**
B.1 Complete Parameter Grids 67
B.2 Correctness and Numerical Checks 67
B.3 Additional Results and Negative Designs 67

C Contributions and Tool-Use Record **68**
C.1 Contribution Record 68
C.2 Generative-AI Assistance 68

D Template Examples — Draft Only **69**
D.1 Citations and Cross-References 69
D.2 A Source-Native Diagram 69
D.3 Tables 69
D.4 Equations and Proofs 69
D.5 Pseudocode and Code 70

List of Figures

5.1 Query execution and its separate selection boundaries. Routing selects lists, primary screening selects vector candidates, and final selection ranks refined survivors. Raw-space ground truth is used for evaluation; it is not an input to these search decisions. 10

5.2 Packed candidate-parallel access, showing a subset of warp lanes. At fixed word group j, consecutive positions access adjacent four-byte words. Each word contains four subspace codes of one candidate. Unpacking preserves each code byte; LUT splitting happens afterwards. The diagram illustrates addresses, not a measured number of memory transactions. 14

5.3 Local coverage and truncated pairwise merging, illustrated with $c = 3$ and $G = 4$. Entries are (D_{P}, p) keys, not measured data. Bold entries are the final global winners. A short group contributes its valid prefix; an empty group contributes no entries. Every merge uses the same key order and retains at most c entries. 17

5.4 Shared complete-key filtering for one scan group. A candidate's full primary score is computed before admission. A valid c-candidate witness set can tighten the per-query bound; a stale but complete bound only weakens filtering. Final prefix trimming reduces the valid merge input, while the allocated workspace capacity remains unchanged. The whole-key-read premise is qualified in the text. . . . 20

5.5 Sixteen-lane residual refinement. A warp handles two candidates; lane a within a candidate reads residual words $a, a+16, \ldots$. With eight-bit residual codes, each 32-bit word decodes four coordinates. Subgroup reductions combine partial scores before adding the retained primary score and correction. Candidate positions can be scattered even when each subgroup's word access is contiguous. 24

7.1 Measured host-to-host recall–QPS frontiers on the four E1 datasets. The displayed recall window begins at 0.90, with a vertical line at 0.97; all baseline families, including CAGRA int8, participate in frontier construction. Lines connect measured points; throughput uses a logarithmic axis. 33
7.2 Cartesian table grouping at fixed $\alpha = 100$ (v71). Group counts describe g_{LUT}, not parallel scan groups. Curves are medians of per-dataset QPS ratios to the one-group arm; shading shows their range. 37
7.3 Packing and refinement controls. Left: median packed-load QPS gain across six datasets, with shaded minimum–maximum ranges, for each table grouping and probe count. Right: measured load-only and 16-lane end-to-end gains at 128 probes and the ordinary dataset-specific budgets. These panels come from different experiment families. 38
7.4 Held-out risk across sample sizes. The vertical axis is the percentage of calibration draws losing more than 10^{-3} held-out recall relative to the best tested budget. Increasing S also tightens the allowed slot-disagreement fraction because equation (7.1) keeps threshold one. 44
7.5 Estimated calibration payback relative to fixed budgets 100 and 64. Filled circles denote 128 probes and open triangles 512 probes. Top-row points have no throughput payback. Estimates invert mean selected-arm QPS and use 500 queries, or 499 for Stella. 47
7.6 Execution regimes. Left: fixed shared-bound group counts and scheduled execution relative to the lane-refined one-block baseline (13 cells at small batches, 11 at batch 1,000). Right: paired scan and merge profiles at batches 8 and 1,000 for arXiv budgets 100 and 200. Hatching denotes merge time. The right panels use the independent-bound scan experiment; they explain a cost, not the scheduler's entire speedup. 48
7.7 Recorded construction costs, with CAGRA int8 included. JHQ is one cold train-plus-add record; other bars are medians across successful measured index configurations, with minimum–maximum whiskers. Int8 includes dataset quantization; RaBitQ excludes the required serialization round trip. Missing large-dataset bars denote unattempted configurations, not measured build failures. The vertical axis is logarithmic. 51

7.8 Resident-memory decomposition at two-group tables, 128 probes and $\alpha = 100$. Component sizes are modelled, markers are measured used totals, and hatching is the unattributed remainder. The totals include context and workspace; construction peaks are not measured. 52
7.9 Large-dataset host-to-host frontiers, including the successful isolated CAGRA int8 reruns. BGE uses 1,000 queries and Stella 999. An endpoint is the end of the tested grid, not an intrinsic recall limit. . . 53

D.1 Formatting example: a schematic query path. Replace or extend it with the verified execution boundaries of the implementation. 69

List of Tables

5.1 Cartesian table trade-offs for an eight-bit, eight-coordinate primary subspace. Combining additions are the algebraic minimum for joining the looked-up values, excluding accumulation across subspaces; they are not measured instruction counts. 12
5.2 Selection stages have different objects, tie rules and reference checks. Evidence for one stage cannot establish another's correctness. 26

7.1 Evidence families. Counts denote measured configurations or checks, not independent index builds. All use the RTX 5090; timing protocols differ. 31
7.2 Recall 0.97 comparison. Each ratio is interpolated / measured; values above one favour JHQ. JHQ kQPS is the interpolated estimate. The small arXiv–RaBitQ difference changes direction with the estimator. . 34
7.3 Refinement at 128 probes. Ratios are paired end-to-end gains over L0, except the last column, which is the profiled refinement-kernel gain. The kernel and application gains must not be interchanged. 38
7.4 v69/v65 end-to-end throughput ratios for the exact FP32 probe change. Each workload–batch comparison has ten timing rounds per binary, collected in alternating runs. Values above one favour v69. 41
7.5 Held-out quality at $S = 128$. Rule α is the median across 64 draws; oracle α uses held-out labels. $\overline{R}_{\text{rule}}$ is reconstructed mean recall. "Losses" counts draws with recall loss $> 10^{-3}$ relative to the best tested held-out recall; L_{95} is the logged lower-order 95th-percentile loss estimate. R_{100} is fixed-budget recall. 43
7.6 Fixed-budget controls over twelve held-out cells. Qualifying means within 10^{-3} of the best tested recall at printed precision. Worst recall loss uses all cells; QPS deficit uses only cells where that arm qualifies. A dash means no qualifying cells. 45
7.7 Calibration economics, $S = 128$. Rates are kQPS and rule rate is mean selected-arm QPS. Ratios compare with fixed 100 and fixed 64. Payback in the last column is relative to fixed 100 using 500 queries, or 499 for Stella; "Never" denotes no throughput payback. 46

7.8 Integrated execution gains over v61 L16. Entries are geometric means of per-cell median paired speedups from five rounds. “Held out” excludes the four configurations used to set the rule; it means held-out configurations, not the budget experiment’s held-out query split. . . . 49
7.9 Cold JHQ construction and separately measured resident memory. Train/add come from one recorded cold build per dataset. Memory comes from v71’s two-group, 128-probe, $\alpha = 100$ runs and includes context and workspace. “Residual” is the modelled residual-code share of the measured used total, not the fraction read per query. 50
7.10 E1 allocation ranges. JHQ is process total minus a 507.2 MiB context estimate, including query workspace; CAGRA fp32 is index allocation measured during build. Ratio ranges combine the extrema for each dataset. 53
7.11 Answers to the four research questions, supported by this chapter. . . 56

A.1 Component and run provenance. “Report” distinguishes a version stated in an experiment report from a raw run-header commit. The exact E1 build state is not inferred from a baseline’s metadata. 64
A.2 Chapter 7 result-to-figure mapping. Paths below are relative to `JHQ_GPU`; detailed file identities are in the hash ledger. 65

D.1 Example manifest fields. This table contains no experimental results. 70
D.2 Pseudocode layout example. 70

Notation and Abbreviations

Symbol	Meaning
N, d	Number of database vectors and original dimension
M, D_s	Number of primary subspaces and dimensions per subspace
k	Number of returned neighbours
n_c, c	Routed candidates and primary survivors receiving refinement
α	Refinement multiplier, with $c = \max\{k, \lceil \alpha k \rceil\}$
S, ϵ	Calibration sample size and allowed disagreeing result slots
g_{LUT}	Number of groups in a factorised lookup table
g_{scan}	Number of scan groups per query
ℓ_{ref}	Number of cooperating lanes per refined candidate
D_{raw}	Squared distance between original vectors
D_{P}	Distance under the primary reconstruction
D_{H}	Refined ranking score: reconstructed squared distance plus a query-only constant

Abbreviation	Meaning
ANN	Approximate nearest neighbour
ADC	Asymmetric distance computation (distinct from the ADC conference)
IVF	Inverted file
JHQ	Johnson–Lindenstrauss enhanced hierarchical quantisation
LUT	Lookup table
PQ	Product quantisation
QPS	Queries per second
GT	Ground truth

TO WRITE: Check every definition against the final implementation and remove symbols that are not used. Keep code parameter names distinct from mathematical notation, especially the multiple uses of G and B.

Chapter 1

Introduction

TO WRITE: Establish the research problem, testable questions, scope, and personal contributions. Keep results consistent with the final evidence manifest.

1.1 Context and Practical Objectives

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

1.2 Problem Definition and Scope

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

1.3 Three Linked Technical Challenges

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

1.4 Research Questions and Falsifiable Claims

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

1.5 Contributions and Research Ownership

TO WRITE: Identify personal work, collaborator roles, inherited algorithms and code, the ADC manuscript relationship, and actual tool assistance.

1.6 Thesis Roadmap

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 2

Background and Critical Related Work

TO WRITE: Explain only background used by the thesis. Compare alternatives critically and distinguish inherited JHQ properties from this work.

2.1 Formal ANN Objective and Metrics

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.2 IVF, PQ, ADC and Residual Refinement

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.3 Inherited JHQ: Rotation, Cartesian Codes and Assumptions

TO WRITE: State that the orthogonal transform is square. Define the primary codebook using its actual formula, residual relative to primary reconstruction, and valid parameters.

2.4 GPU Execution Concepts Used Here

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.5 GPU Graph Search: SONG, CAGRA and BANG

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.6 Quantised ANN, Fast Tables and Modern Alternatives

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

2.7 Critical Synthesis and Precise Research Gap

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 3

Research Design and Measurement Methodology

TO WRITE: Define comparison protocols once. Separate exploratory findings from questions specified before measurement; identify the actual evidence for each claim.

3.1 Research Questions as Controlled Comparisons

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

3.2 Datasets, Queries, Ground Truth and Splits

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

3.3 Measurement Boundary, Variance and Memory Accounting

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

3.4 Baselines, Tuning Grids and Frontier Construction

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

3.5 Versioned Claim-to-Evidence Ledger

TO WRITE: Record commit, binary, compiler options, index/cache identity, query split, raw result, metric, and figure-generation script for every main claim.

3.6 Evidence Strength and Threat Controls

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 4

System Architecture and GPU Index Construction

TO WRITE: Describe build-time dataflow, CPU/GPU responsibilities, physical storage, temporary memory, and representation invariants.

4.1 End-to-End Build and Search Boundaries

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.2 Training, Rotation and Codebook Generation

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.3 IVF Assignment, IDs and Physical Memory Layout

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.4 Resident Storage, Workspace and Index Lifecycle

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.5 Construction Cost and Tested Scale

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

4.6 Index Representation and Safety Contracts

TO WRITE: Distinguish representation checks, algorithmic selection correctness, numerical agreement, and memory-safety checks; do not use recall equality as proof.

Chapter 5

Representation-Aware GPU Search Execution

This chapter addresses RQ2: how JHQ's compressed representation can support correct and efficient GPU execution. The transform, Cartesian primary codebook and residual hierarchy are inherited from JHQ [4]; the focus here is their execution. The argument follows the query path from routing through primary selection to residual scoring. For each mechanism, it separates the mathematical target, implementation conditions and resource trade-offs, with the corresponding measurements in Chapter 7. Workload-level refinement-budget selection is developed separately in Chapter 6.

5.1 Query Path and Four Distinct Ranking Objects

For a fixed index, a query is transformed to $q' = \Pi q$, routed to selected inverted lists, screened using primary codes and refined only on the surviving candidates. The final selector returns k database IDs and their computed scores. Figure 5.1 distinguishes the three runtime selection decisions; the ground-truth ordering is an external evaluation object.

Four quantities must be kept separate:

$$\begin{aligned} s_i(q) &= \|\mu_i\|_2^2 - 2\langle q', \mu_i\rangle, && \text{centroid routing score,} \\ D_{\mathrm{P}}(q,x) &= \|q' - \widehat{p}(x)\|_2^2, && \text{primary score,} \\ D_{\mathrm{H}}(q,x) &= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2, && \text{refined ranking score,} \\ D_{\mathrm{raw}}(q,x) &= \|q - x\|_2^2, && \text{raw-space distance.} \end{aligned} \tag{5.1}$$

Here μ_i is an IVF centroid in transformed coordinates, $\widehat{p}(x)$ is the primary reconstruction and $\widehat{r}(x)$ is the decoded residual. The runtime refined score includes a query-only constant, as derived from the implemented correction in Section 5.6. It

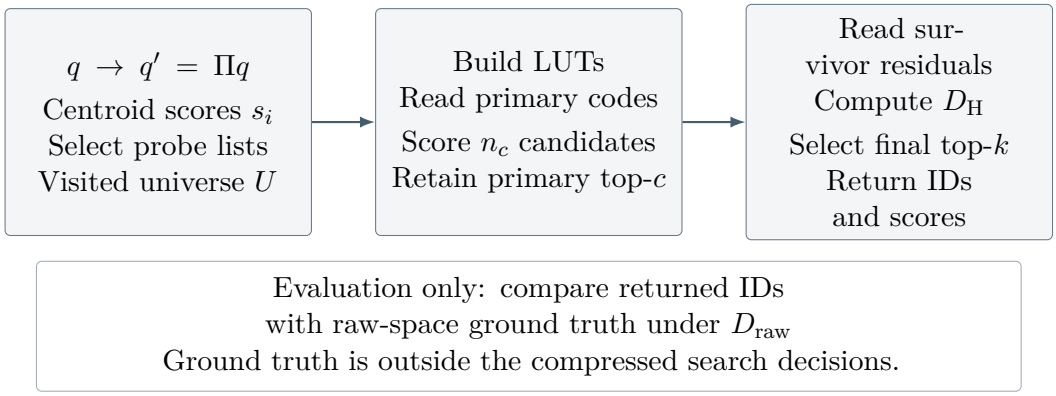


Figure 5.1: Query execution and its separate selection boundaries. Routing selects lists, primary screening selects vector candidates, and final selection ranks refined survivors. Raw-space ground truth is used for evaluation; it is not an input to these search decisions.

preserves the ordering of reconstructed distances for a fixed query, but neither it nor the primary score is the original-vector distance.

Routing computes centroid scores and selects n_{probe} lists (Section 5.7). Concatenating those lists defines the visited universe U of size n_c. An exact probe primitive selects the requested centroids under the computed routing scores; it does not certify that U contains every raw-space neighbour. The prefix offsets also expose the actual candidate counts needed for scanning and scheduling.

Primary screening builds query-specific LUTs and reads primary codes for the visited vectors. It retains up to $c = \max\{k, \lceil \alpha k \rceil\}$ candidates under D_{P}. With fixed table grouping, score evaluation costs $O(n_c M)$, in addition to local-selection and merge work. Sections 5.2 and 5.3 address table and code access; Sections 5.4 and 5.5 establish selection invariants and examine the cost of exposing more parallelism.

Residual scoring then reads residual codes only for valid survivors, performs $O(cd)$ coordinate work and supplies D_{H} to the final selector (Section 5.6). In this path, compressed reconstructions replace search-time raw-vector reads. The implementation's final repeated-minimum selector scans c slots for each of k outputs, with additional block reduction work.

Deterministic ordering keys attach tie rules to these scores: centroid selection uses list ID, complete-key primary selection uses physical row position, and final selection uses database ID. The query-centric legacy primary path has a different cutoff-tie rule, qualified in Sections 5.4 and 5.5. Consequently, score preservation, survivor-set agreement, ordered-output equality and raw-space recall are separate claims. Chapter 7 tests them at their respective boundaries rather than treating a single unchanged recall value as proof of the whole pipeline.

5.2 Exact Cartesian LUT Factorisation

The broad scan evaluates primary reconstructions through asymmetric distance computation: the query remains FP32 while a stored code selects reconstructed coordinates [4, 5]. For transformed query q', the ordinary table for primary subspace m is

$$T_m[z] = \|q'_m - C_m[z]\|_2^2, \qquad D_{\mathrm{P}}(q, x) = \sum_{m=0}^{M-1} T_m[z_m(x)]. \tag{5.2}$$

With an eight-bit code, this requires 256 entries per subspace. The opportunity here comes from the codebook's internal Cartesian structure, not just the fact that the vector is partitioned into PQ subspaces.

5.2.1 Factorisation without changing the quantiser

For the tested $D_s = 8$ Cartesian codebook, the high nibble h selects the first four reconstructed coordinates and the low nibble l selects the remaining four. Thus $C_m[16h + l] = (A_m[h], B_m[l])$, with no cross-dependence between the two coordinate groups. Define

$$T_m^{\mathrm{hi}}[h] = \|q'_{m,\mathrm{hi}} - A_m[h]\|_2^2, \qquad T_m^{\mathrm{lo}}[l] = \|q'_{m,\mathrm{lo}} - B_m[l]\|_2^2. \tag{5.3}$$

Squared-L2 additivity over disjoint coordinates gives

$$T_m[z] = T_m^{\mathrm{hi}}[z \gg 4] + T_m^{\mathrm{lo}}[z \,\&\, 15]. \tag{5.4}$$

Only 32 entries are stored, rather than 256. This preserves the reconstruction and the real-arithmetic score; no code retraining, extra quantisation or change in the candidate universe is involved.

The builder reads representative codewords $16h$ and l from the existing full codebook and evaluates only their corresponding coordinate slices. The zero digits in the other slice are irrelevant. This is valid only when all codewords sharing a nibble share that slice. The `verify_split_lut` check compares the stored codeword slices against their representatives and rejects incompatible codebooks. An arbitrary learned 256-centroid PQ subcodebook need not satisfy this condition: dividing its code labels into nibbles does not factor its centroid coordinates.

The mathematical identity does not promise bitwise FP32 equality. A full eight-coordinate accumulation and two four-coordinate sums followed by an addition

Table 5.1: Cartesian table trade-offs for an eight-bit, eight-coordinate primary subspace. Combining additions are the algebraic minimum for joining the looked-up values, excluding accumulation across subspaces; they are not measured instruction counts.

g_{LUT}	Entries	FP32 bytes	Lookups	Combining adds	Build terms
1	256	1,024	1	0	2,048
2	32	128	2	1	128
4	16	64	4	3	32
8	16	64	8	7	16

associate the operations differently. The implementations therefore retain FP32 tables and separately check numerical and ranking effects. Replacing table entries with FP16 would introduce another rounding operation and is a different design choice.

5.2.2 Grouping, construction work and resource cost

The v71 ablation generalises the split to $g = g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$ contiguous coordinate groups whose bit fields match the Cartesian digits. For eight bits and $D_s = 8$, each group has $2^{8/g}$ entries; the total entries and FP32 storage per query are

$$E(g) = g2^{8/g}, \qquad W_{\mathrm{LUT}}(g) = 4ME(g) \text{ bytes.} \tag{5.5}$$

Independent entry construction evaluates $D_s 2^{8/g}$ coordinate contributions per subspace. Two groups therefore reduce table storage eightfold and coordinate work sixteenfold relative to independently constructing every full-table entry. These are table-level cost ratios, not predicted application speedups.

The table is first built in global workspace. A scan block stages it cooperatively in shared memory when the table and that kernel's candidate buffer fit the device's configured limit; a block barrier completes staging before lookup. Otherwise it reads the global table. The shared footprint must include selection scratch: W_{LUT} alone does not determine residency. Grouped scan blocks may each stage the same query's table, coupling this design to Section 5.5.

At $M = 96$ and 384, full FP32 tables require 96 and 384 KiB per query; two-group tables require 12 and 48 KiB. The smaller footprint creates placement opportunities, but four and eight groups make more lookups and combine more values even though both store only 16 entries per subspace. Index extraction, dependent lookups, synchronisation and residency all contribute to the resulting trade-off. Table size

alone cannot identify the fastest design or establish a unique cache/bank-conflict explanation.

Small-table execution has substantial precedent in PQ Fast Scan and Quicker ADC [1, 2]. This implementation applies exact coordinate factorisation to JHQ's existing Cartesian codebook and measures the GPU grouping trade-off; it does not claim to invent small LUTs. Section 7.3 reports the controlled v71 sweep: two groups lead in 22/24 cells, with the other two differences below 1%. That result supports the selected operating point rather than a universal optimum.

5.3 Packed Codes and Candidate-Parallel Memory Access

LUT factorisation reduces the query-specific table; code layout determines how lanes fetch each candidate's table indices. These are separate optimisations. The scan assigns neighbouring lanes to neighbouring candidate occurrences, so its broad phase benefits from storing codes by subspace across candidates rather than storing each candidate's full primary code contiguously.

5.3.1 Address mapping and four-byte packing

Let o_ℓ be the start of inverted list ℓ, r the row within that list and $p = o_\ell + r$ its physical position. If $P[m, p]$ denotes the eight-bit code for primary subspace m, the byte-oriented scan copy stores it at byte offset $mN + p$. Candidate-major storage would instead use $pM + m$. When lanes visit consecutive positions within a list, the former makes their accesses at fixed m adjacent. At list boundaries, positions can jump, so the mapping does not guarantee one contiguous transaction for every warp.

The packed representation combines four subspace codes of the same candidate into one unsigned word:

$$W[j, p] = \sum_{t=0}^{3} 2^{8t} P[4j + t, p], \qquad \text{offset}_{\text{bytes}}(W[j, p]) = 4(jN + p). \tag{5.6}$$

The four codes are recovered exactly by $(W[j, p] \gg (8t)) \,\&\, 255$. Each recovered code then supplies the high and low nibble indices in equation (5.4). Packing four subspaces is distinct from Cartesian splitting of one subspace's eight-bit label, and from packing codes for four different candidates.

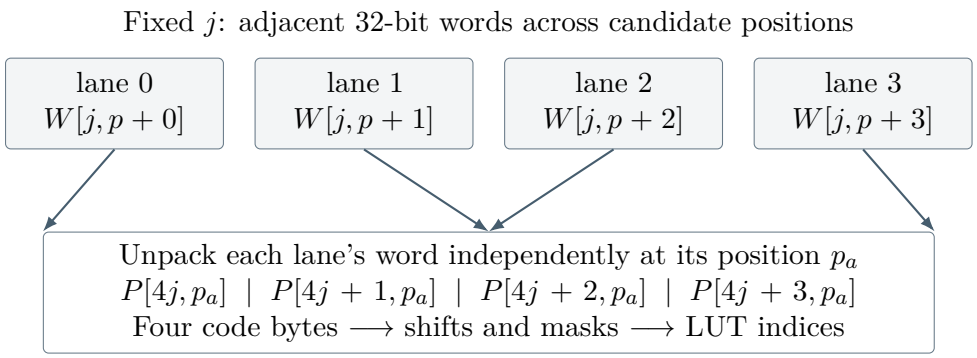


Figure 5.2: Packed candidate-parallel access, showing a subset of warp lanes. At fixed word group j, consecutive positions access adjacent four-byte words. Each word contains four subspace codes of one candidate. Unpacking preserves each code byte; LUT splitting happens afterwards. The diagram illustrates addresses, not a measured number of memory transactions.

The build path explicitly repacks the $[M, N]$ byte array into $[M/4, N]$ words. Merely reinterpreting the original byte array as `uint32_t` would combine consecutive candidate bytes rather than the required subspaces. The implementation requires M divisible by four and rejects a partial final word. The repack reads and writes NM code bytes once at construction, with old and new storage briefly coexisting; it is outside the search timing boundary.

5.3.2 What packed access saves, and what it preserves

At the source level, a candidate now obtains four codes with one word load instead of four byte loads, followed by shifts and masks in registers. Within a contiguous list segment, 32 lanes request a logical 128-byte word window. Actual transactions depend on alignment, active lanes and the memory hierarchy. A fourfold reduction in source-level load operations does not establish a fourfold reduction in transactions or elapsed time.

The logical payload remains M bytes per candidate: this repacking does not further compress the index. Both byte and word alternatives already organise the scan copy by subspace or subspace group. The historical on/off comparison therefore measures the net effect of the packed layout and its load/unpack path, not the isolated contribution of transposing candidate-major storage. It also cannot uniquely separate instruction count, addressing and cache effects.

Section 7.3 reports 48 matched byte/word configurations with throughput ratios 1.012–1.658×. Each arm has one aggregate rate, so these records provide weaker variability evidence than the newer repeated LUT sweep. The integer representation

identity is exact, while the observed recall spread of at most 10^{-4} belongs to the complete measured execution. The method claim is preservation of code values with fewer load/address operations, qualified by the experiment's measurement scope.

5.4 Exact Global Primary Top-c via Multi-Block Scanning

Primary screening must expose enough GPU work without losing a candidate that should reach residual refinement. A query-centric scan launches one block per query; a small batch therefore offers few blocks even when each query visits many vectors. GPU selection is a substantial design problem in its own right [6]. The grouped-scan component, introduced in v62, partitions the visited candidates across blocks and combines their survivors. Its contract is exact selection under the computed primary scores. IVF routing and the quantised representation remain approximate.

5.4.1 Candidate partition and deterministic ordering

For a fixed query, concatenate the selected inverted lists in probe order to obtain n_c candidate occurrences. Assume that each visited database row occurs once and that the index stays fixed during search. For $G = g_{\text{scan}} \in \{1, 2, 4, 8\}$, block (q, g) owns the interval

$$U_g = \left[\left\lfloor \frac{g n_c}{G} \right\rfloor, \left\lfloor \frac{(g+1) n_c}{G} \right\rfloor \right), \qquad g = 0, \ldots, G-1. \tag{5.7}$$

Here an interval denotes the corresponding candidates, rather than their database IDs. The device computes its endpoints from the routed candidate count. Groups differ in size by at most one candidate; a long list can cross a group boundary. This balances candidate counts, whereas assigning equal numbers of lists need not do so. Each thread finds its first list through the probe-prefix offsets and advances its list cursor while scanning its interval. Residual codes are not read at this stage.

The selection key is

$$K_q(x) = \Big(D_{\mathrm{P}}(q, x), p(x) \Big), \qquad (a, p) \prec (b, r) \iff a < b \text{ or } (a = b \text{ and } p < r), \tag{5.8}$$

where $p(x)$ is the physical row position in the fixed inverted-list storage. It is unique among the query's candidates and independent of the order of atomic appends. It is not the database ID used to break final-result ties. With non-NaN scores, this defines

a total order on distinct candidates. All local comparisons and merge decisions use that same order. Partition-independent selection additionally requires that each candidate receive the same computed score in every compared path; the ordering rule alone cannot remove floating-point scoring differences.

5.4.2 Exact local retention and global coverage

Write $\mathrm{Top}_c(V)$ for the first $\min(c, |V|)$ candidates of V sorted by K_q, with $c = \max\{k, \lceil \alpha k \rceil\}$. Each independent group retains its full local top-c, not a share c/G. This distinction is necessary: with $c = 4$ and two groups, all four global winners may belong to one group. Retaining only two from each group loses winners even though the combined capacity is four.

Proposition 5.1 (Local coverage and truncated merging)**.** *For disjoint groups $U_0, \ldots, U_{G-1}$, fixed candidate scores and a common total order,*

$$\mathrm{Top}_c\left(\bigcup_g U_g\right) = \mathrm{Top}_c\left(\bigcup_g \mathrm{Top}_c(U_g)\right). \tag{5.9}$$

Pairwise sorted merging with truncation to c after every merge preserves this result, including when fewer than c candidates exist.

Proof. Let x belong to the global top-c and to group U_g. Fewer than c candidates precede x globally, so fewer than c precede it within U_g. Consequently $x \in \mathrm{Top}_c(U_g)$, and the local union contains every global winner. Selecting its first c therefore recovers precisely the global winners in the same order. If the whole universe has fewer than c members, every member is retained.

For merging, the first c of two sorted lists are their union's top-c. A discarded candidate already has at least c predecessors within that union and cannot enter the top-c of any larger union. Induction over the merge tree establishes the claim. □

The CUDA implementation maintains a shared-memory buffer of C key pairs, where C is a power of two and $C \geq c + T$ for a scan block of T threads. A compaction sorts this buffer with a bitonic network, retains at most c valid entries and records the last retained key as the local threshold when c entries exist. Before that point the threshold is a sentinel above every valid key. Between compactions, only candidates preceding the threshold are appended. Its c witness candidates make the threshold a safe upper bound for the best c of the scanned prefix, even as further candidates arrive.

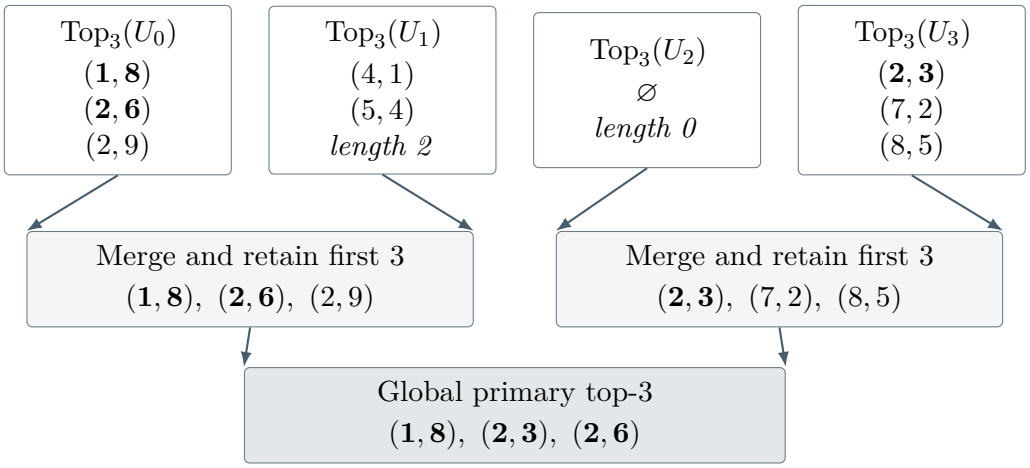


Figure 5.3: Local coverage and truncated pairwise merging, illustrated with $c = 3$ and $G = 4$. Entries are (D_{P}, p) keys, not measured data. Bold entries are the final global winners. A short group contributes its valid prefix; an empty group contributes no entries. Every merge uses the same key order and retains at most c entries.

Each chunk contributes at most T appends. After a chunk, the block compacts whenever the next chunk would exceed capacity. Following compaction at most c entries remain, so $C \geq c + T$ provides space for the next chunk; otherwise the explicit capacity check provides it. This invariant prevents overflow without restricting how many competitive candidates one thread may encounter. A per-thread fixed-size retention buffer does not provide the same coverage guarantee.

An alternative is to materialise all n_c score/position pairs and then invoke a separate selector. That preserves the target but adds an $O(n_c)$ intermediate array and its writes. Here packed-code scoring feeds a bounded local buffer directly, exchanging repeated local sorting for $O(Gc)$ intermediate output. Register-based selection is another established GPU design [6]; this campaign evaluates the chosen scan variants rather than demonstrating superiority over every library selector.

5.4.3 Merge implementation, storage and boundary cases

For $G > 1$, one 256-thread block per query performs $\log_2 G$ rounds of pairwise merging. A pair with valid lengths a, b emits $\min(c, a + b)$ keys. Merge Path partitions this output into contiguous thread slices: a binary search finds the input offsets for each slice, then the thread merges its slice sequentially [3]. All comparisons use equation (5.8); block barriers separate rounds. Figure 5.3 illustrates the truncation invariant. The tree emits at most $c(G - 1)$ key pairs across its internal nodes, in addition to the partition searches and synchronisation work.

The two global score/position workspaces have capacities $[b_{\text{cap}}, G, c]$ and $[b_{\text{cap}}, G/2, c]$ and alternate between rounds. With four-byte scores and positions, their combined payload is

$$W_{\text{pairs}} = 8 b_{\text{cap}} c \left(G + \frac{G}{2} \right) \quad \text{bytes}, \qquad G > 1. \tag{5.10}$$

Group counts require another $4 b_{\text{cap}} G$ bytes; shared bounds add $8 b_{\text{cap}}$ when provisioned. These are additional to final survivor arrays and other search workspace. Allocation grows before graph capture and is retained for reuse, so choosing a smaller G later does not necessarily release the reserved memory. The merge keeps only list lengths in shared memory rather than all Gc keys.

Counts, rather than sentinel scores alone, delimit local valid prefixes. Compaction pads its sorting buffer with $(+\infty, \text{INT_MAX})$; valid positions must be below that reserved position. Empty groups have count zero. The final survivor array pads unused slots with $(+\infty, -1)$, and refinement skips negative positions. These two sentinels serve different purposes: the negative padding position must not participate as a valid key in a merge. $G = 1$ writes directly to the survivor arrays and omits the merge.

The older query-centric path is selected by the implementation label $G = 0$, meaning one block per query, not zero blocks. Its default primary selection is exact by score, but its distance-only cutoff can retain a different member of a tie from the complete-key path. The lossy `K_LOCAL=4` alternative requires `JHQ_EXACT_TOPCK=0` in a build that permits it; paper-faithful builds force exact selection. It is not the measured default. Section 7.4 reports 64 grouped-scan comparisons with no non-tie primary differences and identical final outputs on the used queries. Those checks support the implementation within their coverage. The proposition establishes a conditional selection property, not raw-space exact kNN or unconditional equivalence to every legacy tie policy.

5.5 Shared Complete-Key Thresholds and Execution Scheduling

Full local top-c retention establishes correctness, but can produce Gc intermediate survivors for only c global outputs. The shared-bound component (v63) reduces this excess selection work. The scheduler (v65) then chooses how much scan parallelism

to expose. Both operate at a fixed refinement budget; choosing that budget is the separate problem in Chapter 6.

5.5.1 A complete-key upper bound with explicit witnesses

Let $K_{(c)}$ be the global cth key when $|U| \geq c$. If a group holds c distinct, fully scored candidates, the largest key among those witnesses, τ_g, satisfies $K_{(c)} \preceq \tau_g$. There are already c candidates no later than τ_g in the global universe. The shared bound is initialised to a top sentinel and updated by

$$\tau \leftarrow \min_{\prec}(\tau, \tau_g). \tag{5.11}$$

No bound is published from fewer than c valid candidates. If the entire universe has fewer than c, no group can publish such a bound and all valid candidates remain eligible.

Proposition 5.2 (Safe rejection by a witnessed bound)**.** *Assume fixed keys, publications backed by c distinct candidates, and reads that return either the initial sentinel or one complete published key. Rejecting a candidate only when $K_q(x) \succ \tau$ cannot remove a global top-c candidate, irrespective of publication timing.*

Proof. Every published bound is at least $K_{(c)}$, and the minimum of such bounds is also at least $K_{(c)}$. A global winner satisfies $K_q(x) \preceq K_{(c)} \preceq \tau$, so it is never rejected. An older bound remains an upper bound and merely permits extra work. Publications made after earlier safe rejections remain valid: any c retained distinct candidates still supply c witnesses, whether or not they are the best c of their group. □

This last distinction matters. With sharing enabled, a group need not return its own full local top-c: it can discard locally competitive candidates already certified as globally uncompetitive. Correctness requires the union still to contain every global winner. The truncated merge argument of Proposition 5.1 then applies to that union. Candidates equal to the shared bound remain eligible, including the witness that established it. The implemented admission condition is

$$K_q(x) \prec \lambda_g \quad \text{and} \quad K_q(x) \preceq \tau_{\text{seen}}, \tag{5.12}$$

where λ_g is the local compaction threshold and τ_{seen} is the group's last observed shared bound.

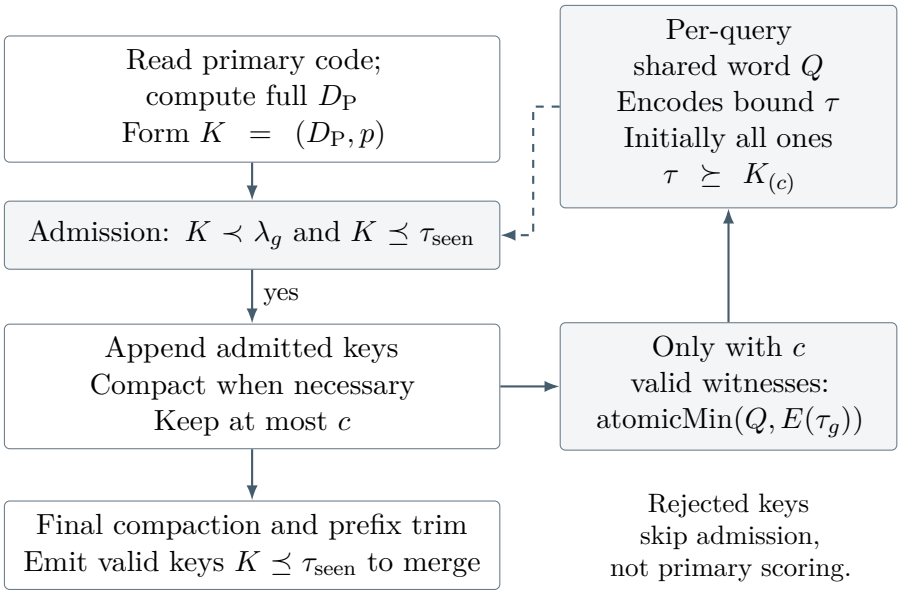


Figure 5.4: Shared complete-key filtering for one scan group. A candidate's full primary score is computed before admission. A valid c-candidate witness set can tighten the per-query bound; a stale but complete bound only weakens filtering. Final prefix trimming reduces the valid merge input, while the allocated workspace capacity remains unchanged. The whole-key-read premise is qualified in the text.

5.5.2 Publication, filtering and implementation limits

For nonnegative FP32 scores generated as sums of squares, the code packs the distance bits and nonnegative position into an unsigned 64-bit key:

$$E(D, p) = 2^{32}\,\mathrm{uintbits}_{32}(D) + p. \tag{5.13}$$

Unsigned integer comparison then agrees with the lexicographic order. This relies on positive-zero encoding and excludes NaNs and negative scores; it is not a general signed-float encoding. Positive infinity orders after finite scores, and the all-one initial word orders after valid encodings. Invalid positions are excluded from bound publication. These are score-domain and representation preconditions, not properties inferred from unchanged recall.

One thread per block reads the query's bound for each chunk. After compaction, it publishes the local cth key through 64-bit `atomicMin` when enough valid entries exist. At scan completion, another publication and a binary search retain only the sorted prefix no later than the observed bound. There is no cross-block barrier or wait for other groups. A late update can leave extra entries in another group's

output, which the exact merge safely removes. Bounds are reset before every search in the same stream, and the merge runs after the scan kernel finishes.

The read premise deserves a separate qualification. The archived code publishes atomically but polls through `volatile` 64-bit loads. CUDA does not guarantee atomicity or inter-thread synchronisation for `volatile` accesses [7]. Portable realisation of the proposition requires device-scope atomic reads as well. The recorded 48 paired checks in Section 7.4 support observed agreement for the tested build, not a language-level concurrency proof. An atomic-read revision and its GPU revalidation have not been performed.

Figure 5.4 also identifies what this optimisation saves. Every scanned candidate still loads its primary code and computes its complete primary score. The bound avoids some buffer appends, local sorts and intermediate writes, and shortens the lists consumed by the merge. It does not implement partial-distance early termination or reduce the number of primary scores evaluated. Its benefit competes with polling and atomic-update overhead. The observed paired speedup range includes a slight slowdown, as reported in Section 7.4.

5.5.3 Choosing parallelism without changing the budget

For batch size b, grouped scanning launches bG blocks instead of b. This can expose otherwise unavailable parallelism, but duplicates table staging and local selection and introduces a merge. The fixed-group design therefore cannot be uniformly preferable. Smaller static quotas would reduce intermediate storage, but require a certificate or fallback to preserve coverage; the pilot's concentrated survivor distribution motivated using witnessed thresholds instead (Section 7.4).

The implemented scheduler uses a fill proxy and a workload-skew statistic. Let T be scan threads per block, H the number of SMs times the maximum resident threads per SM, and $v = \mathrm{sd}(n_c)/\,\mathrm{mean}(n_c)$ the candidate-count coefficient of variation. Its rule is

$$G_{\mathrm{fill}} = \max(\{1\} \cup \{g \in \{2,4,8\} : bgT \le 0.6H\}), \tag{5.14}$$

$$G_{\mathrm{skew}} = \begin{cases} 4, & v \ge 0.3, \\ 1, & v < 0.3, \end{cases} \qquad G_{\mathrm{chosen}} = \max(G_{\mathrm{fill}}, G_{\mathrm{skew}}). \tag{5.15}$$

The skew rule may override the fill limit. In the evaluation harness, v is calibrated using 512 database vectors as pseudoqueries and a fixed routing configuration; it uses

no ground-truth labels. This is distinct from Chapter 6's sampled output-agreement calibration. Once calibrated, scheduling uses the batch size and stored statistic without an additional per-search host readback. The constants were fitted on four pilot configurations and then evaluated on further configurations.

When $G_{\text{chosen}} = 1$, the implementation selects the faster query-centric path, labelled $G = 0$. It leaves the index, probe count and refinement budget unchanged, but inherits that path's distance-only cutoff ties. The common-total-order premise must therefore be checked before claiming identical survivor sets across all scheduler choices. Section 7.6 reports the separate integrated-output checks and the scheduler's measured gains and regret.

The rule is a heuristic rather than an occupancy calculation or a latency oracle: it omits register pressure, shared-memory residency, budget-dependent compaction cost and merge cost. A frozen pseudoquery statistic can also misrepresent future queries. In the recorded campaign the largest regret occurs at batch 256, where a fixed-group alternative is about 1.217× faster. Thus the design contribution is a tested combination of parallel candidate scanning, witnessed filtering and workload-dependent execution, not a new universal top-c identity or a guarantee of optimal scheduling. Refinement and final top-k consume the resulting valid survivors as described in Section 5.6.

5.6 Lane-Cooperative Residual Scoring and Final Top-k

Only the primary survivors access residual codes. This preserves a broad, cheap scan over n_c candidates followed by a narrower $O(cd)$ scoring stage. Reading residuals for every visited candidate would instead move that coordinate work towards $O(n_c d)$. Lane cooperation accelerates the chosen survivors; it does not decide the refinement budget.

5.6.1 Reconstructed scoring and the query-only constant

Let $\widehat{p}(x)$ be the primary reconstruction and $\widehat{r}(x)$ the decoded residual in transformed coordinates. The encoder stores $\gamma(x) = 2\langle \widehat{p}(x), \widehat{r}(x) \rangle$. The implemented refined

ranking score is

$$D_{\mathrm{H}}(q,x) = D_{\mathrm{P}}(q,x) + \|q' - \widehat{r}(x)\|_2^2 + \gamma(x) \tag{5.16}$$

$$= \|q' - \widehat{p}(x) - \widehat{r}(x)\|_2^2 + \|q'\|_2^2. \tag{5.17}$$

The second equality follows by expanding the squared norms. Thus the score orders hierarchical reconstructions by squared distance, but the returned value includes a query-only constant. It is neither a direct raw-vector distance nor an unshifted reconstruction distance. The constant does not affect a fixed query's ordering. Reconstruction error remains even when selection under this score is exact [4].

Residual coordinates use scalar codebook entries shared within a subspace. The scalar path walks each candidate's dimensions and sums their squared differences from q'. Query coordinates and, when space permits, the residual codebook are staged in shared memory; the stored correction and primary score are added afterwards.

5.6.2 Mapping lanes to words and reducing partial sums

For $\ell_{\mathrm{ref}} = L \in \{1, 8, 16, 32\}$, a scan-independent refinement block assigns L adjacent lanes to a candidate. A 32-bit residual word holds $R = 32/B_R$ coordinate codes, where B_R is the residual bit width. The tested eight-bit path has $R = 4$; the four-bit path has $R = 8$. Lane a reads word indices $a, a + L, a + 2L, \ldots$, decodes their coordinates and accumulates a partial sum. A butterfly shuffle reduction combines the L partials, and lane zero writes the refined score.

The launch requires a word-aligned residual row and D_s divisible by R, so a word never crosses a subspace's codebook boundary. All lanes execute the shuffle sequence, including lanes assigned to invalid or out-of-range survivor slots; these contribute zero before the output is marked infinite. This avoids making warp participation depend on whether a particular candidate exists. At $L = 16$ a warp handles two candidates, rather than one candidate per lane.

The experiment separates two changes. L0 is the original scalar byte-load path; L1 uses word loads with one lane per candidate, preserving its accumulation order. L8/L16/L32 additionally distribute coordinate work across lanes. This shortens each lane's serial walk and provides useful work when c is small, but adds shuffles and reduces the number of independent candidates handled concurrently. Larger L is therefore not automatically better. Irregular survivor positions and shared/global codebook placement remain relevant costs.

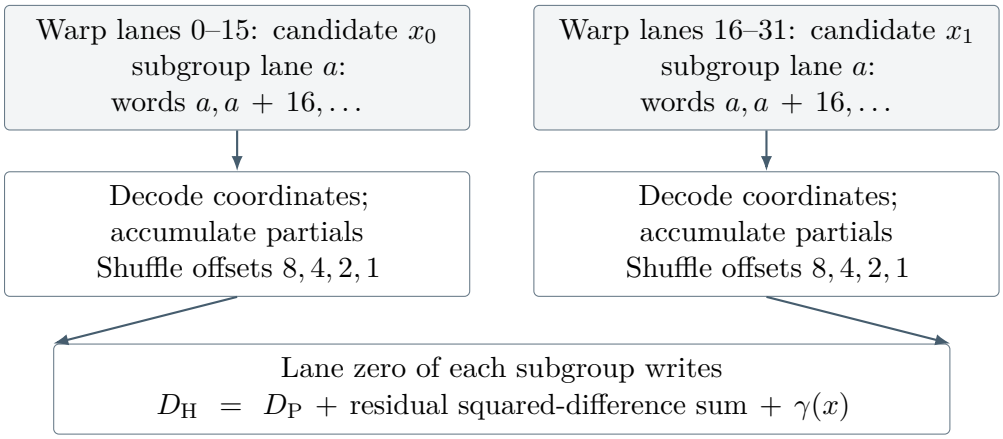


Figure 5.5: Sixteen-lane residual refinement. A warp handles two candidates; lane a within a candidate reads residual words $a, a + 16, \ldots$. With eight-bit residual codes, each 32-bit word decodes four coordinates. Subgroup reductions combine partial scores before adding the retained primary score and correction. Candidate positions can be scattered even when each subgroup's word access is contiguous.

5.6.3 Final selection and the numerical boundary

For at least k valid survivors with finite refined scores and unique database IDs, the final kernel performs exact top-k selection under $(D_{\mathrm{H}}, \text{database ID})$. One block per query loads the c scores and repeatedly reduces the minimum, emits its ID and score, and removes that score before the next round. All c slots are scanned at each round; there is no fixed per-thread candidate quota. This correctness statement does not establish the final-output contract for degenerate pools with fewer than k finite valid candidates.

The final ID tie rule differs from the physical-position rule used for primary retention. An exact final selector cannot recover candidates discarded by routing or screening, nor remove quantisation error. Wider lane reductions also change floating-point association before selection, which can move near-tied candidates even when the real-arithmetic formula is unchanged.

Section 7.3 separates these effects: L1 is bit-identical in its tested cases, while multi-lane distance differences reach approximately 1.6×10^{-6} relative error. The ordinary L16 end-to-end gains range from 1.015 to 1.789×; kernel-only gains are larger and must not be substituted for them. The design is supported by controlled load-only and cooperative variants, with numerical equivalence tested rather than assumed.

5.7 IVF Probe Selection as an Independent Primitive

Probe selection precedes primary scanning and selects inverted lists, not database-vector survivors. For IVF centroid μ_i, the selector orders the computed FP32 routing scores

$$s_i = \|\mu_i\|_2^2 - 2\langle q', \mu_i \rangle, \qquad (s_i, i) \text{ ordered lexicographically.} \tag{5.18}$$

Omitting the common query norm preserves centroid ordering. In contrast to primary squared-distance tables, these scores can be negative. The required output is the first $P = \min(n_{\text{probe}}, n_{\text{list}})$ centroid IDs in score order, using the smaller list ID at a tie, plus the prefix offsets of their list lengths.

5.7.1 Radix selection followed by deterministic ordering

The v65 selector repeatedly bitonic-sorted a growing candidate buffer. The v69 replacement keeps one block per query but finds the cutoff by four most-significant-byte-first radix passes. Each pass counts only keys matching the prefix selected so far, then locates the bin containing the remaining order statistic. After four passes it has the cutoff key and the number of equal-cutoff entries still required.

The float mapping canonicalises both signed zeros to positive zero, then transforms their 32-bit representation u by

$$F(u) = \begin{cases} \sim u, & \text{if the sign bit is set,} \\ u \mid 2^{31}, & \text{otherwise.} \end{cases} \tag{5.19}$$

For non-NaN floats this preserves numerical order, including negative scores and infinities; canonicalisation makes equal zeros share a key. This differs from the nonnegative-score encoding in equation (5.13). NaNs are outside the selector's ordering contract. Every pass recomputes scores with the same explicitly rounded `fma` expression, so changing the pass does not change its keys.

The histogram has 256 bins and 32 lane replicas, requiring 32 KiB for its counters alone. Replicas separate same-warp atomic updates to a popular bin; different warps can still contend. This exchanges shared storage and reduction work for less concentrated histogram updating, rather than eliminating all contention.

Table 5.2: Selection stages have different objects, tie rules and reference checks. Evidence for one stage cannot establish another's correctness.

Stage	Ordering object	Tie key	Relevant evidence
Probe	Centroid routing score	List ID	Synthetic IDs, offsets and totals; real-input dumps
Primary	Visited-candidate primary score	Physical position in grouped path	Local/merge proof; independent/shared checks
Final	Refined survivor score	Database ID	Full-c selection code; residual/output comparisons

Collection first gathers every key below the cutoff. If too many keys equal the cutoff, an ordered rescan uses ballots and prefix counts to take exactly the required number in increasing list-ID order. Atomic arrival order therefore does not choose the final tied subset. For each of the P selected entries, a rank-count step counts how many selected keys precede it under (s_i, i) and writes it to that rank. This restores deterministic sorted order without sorting all centroids.

Finally, a block prefix scan produces offsets of the selected list lengths and their total candidate count. When $n_{\text{probe}} > n_{\text{list}}$, the implementation pads the ID array by repeating the final selected ID but assigns those extra entries zero length. They do not scan the same list again. Nonempty centroid sets and a positive requested probe count are assumed.

5.7.2 Costs and separate correctness contracts

The histogram passes and collection revisit the centroid-score row; rank counting costs $O(P^2)$ comparisons. The algorithm is consequently not a uniformly linear-cost substitute for every probe count. Large P increases both ranking work and selected-entry shared storage, while the histogram reserves space even for small centroid sets. Radix selection is an established selection technique; the contribution here is its deterministic integration and measured execution trade-off for this query pipeline.

The archived probe checker reports 1,792 cases with zero differences against the earlier selector in IDs, offsets and total candidate counts. Its distributions include many ties, all-equal values, mixed signed zeros, mostly infinite values and extreme magnitudes. Cases also cover requested probe counts larger than the number of lists. These are targeted finite tests of the probe primitive, not a proof of primary top-c or of exact nearest neighbours.

Section 7.4 gives the real-query identity checks, stage timings and end-to-end comparison. It also retains the Vogue cross-process anomaly: matching probe/primary dumps do not imply identical final pipeline outputs. The optional v70 FP16 routing changes score precision and is a separate approximation; high probe overlap or unchanged recall does not make it part of the FP32 equivalence claim.

Chapter 6

Workload-Level Refinement-Budget Calibration

TO WRITE: Define the policy, its reference outputs and assumptions, then separate sampled agreement, held-out quality, and calibration economics.

6.1 Why Fixed Refinement Budgets Can Be Wasteful

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

6.2 GT-Free Sampled Output Agreement

TO WRITE: Define the disagreement-slot sum and distinguish it from a per-query rate. Explain how fixed epsilon changes strictness as the sample size changes.

6.3 Budget Search, Nestedness and Tie Safety

TO WRITE: Check nested survivor sets, fixed refined scores, exact-selection configuration, deterministic ties, and the executed path before invoking monotonicity.

6.4 Budget Baselines and Their Questions

TO WRITE: Explain the distinct questions answered by fixed budgets, same-sample enumeration, full-pool calibration, and the ground-truth oracle.

6.5 Hold-out Risk, Calibration Cost and Payback

TO WRITE: Separate selected-arm throughput from total cost. State held-out quality loss, sampling variability, calibration overhead, and reuse assumptions.

6.6 Policy Failure Modes and Applicability

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 7

Experimental Evaluation and Research Findings

This chapter answers the thesis's four research questions with measurements of construction, controlled execution changes, held-out budget selection and cross-system search. For **RQ1** (construction and capacity), the largest tested index searches 17.78 million vectors using 20.44 GiB of resident device memory; its recorded cold train-plus-add time is 10.40 s. For **RQ2** (correct and efficient GPU execution), two-group tables lead in 22/24 configurations, and scheduled scanning improves geometric-mean throughput by 2.727× at batch 8 while preserving tested final ID sets. For **RQ3** (budget quality and net cost), mean selected-arm QPS beats fixed $\alpha = 100$ in 10/12 cells and fixed $\alpha = 64$ in 9/12, but up to 40/64 calibration draws exceed the oracle-relative recall-loss threshold. For **RQ4** (competitive operating regions), at recall 0.97 JHQ is fastest in three E1 datasets and numerically close to IVF-RaBitQ on arXiv under the interpolated comparison; CAGRA int8 leads at lower recall.

7.1 Final Experiment Manifest and Comparison Families

The experiment families separate the evidence needed for RQ1–RQ4. Component names describe the mechanism; implementation versions identify its provenance. Recorded run heads, binary names and source-to-result mapping are given in Chapter A.

7.1.1 Evidence families and measurement boundaries

Table 7.1 identifies the experiment families used in this chapter. Version numbers name experimental implementations, not successive measurements of one unchanged benchmark. In particular, the historical ADC paper and the later v69/v71 runs have different timing results. The newer raw records determine the corresponding num-

Table 7.1: Evidence families. Counts denote measured configurations or checks, not independent index builds. All use the RTX 5090; timing protocols differ.

Family	Purpose and implementation	Coverage
E1	v69 versus cuVS graph, PQ, RaBitQ and brute force	1,590 points; 4 datasets
LUT	v71 table grouping, $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$	24 cells; 96 arms
Packing	Historical byte/word access factorial	48 paired cells
Refinement	v61 scalar/load/lane ablation	13 configurations
Scan/bound	v62 global selection; v63 shared bound	64 / 48 correctness checks
Scheduling	Integrated v65 execution policy	50 workload–batch cells
Probe	v69 exact FP32 probe selection	1,792 synthetic cases
Budget	v69 sampled, full-pool and oracle budgets	12 cells; 64 draws per S
Scale	v69 search, cold builds and resident memory	BGE and Stella extensions

bers here; historical packing and refinement experiments remain separate ablations. Multiplying their speedups would combine different baselines and would not yield a measured system speedup.

The E1 environment records an NVIDIA GeForce RTX 5090 with 32,607 MiB reported device memory, driver 595.71.05, CUDA 13.0 and cuVS 26.08.01. The container has a 25-CPU quota; OpenMP is limited to 16 threads. JHQ uses Release code compiled to compute_80 PTX and JIT-compiled for the device; the baselines use the installed native cuVS libraries.

The principal cross-system timing boundary starts with FP32 queries in host memory and ends after top-10 identifiers and distances return to host memory and the stream synchronizes. It includes transfers, query preprocessing and search; CAGRA int8 query quantization is inside this boundary. Baselines reuse resource handles and output buffers. E1 uses five timing rounds, normally 24 calls per round. The baseline harness reduces this count when the warm-up interval, including the first call, exceeds its per-call threshold; 274 points use fewer calls, including some fast steady-state points. Its reported QPS is the median of round throughputs, each computed from the total queries and total elapsed time. It is not the reciprocal of the median individual-call latency. Warm-up, index construction and any budget calibration are excluded. Device-only baseline times are auxiliary: JHQ has no equivalent device-query API, so mixing those times into the main ratios would change the comparison.

7.1.2 Metrics and reproducibility

All experiments use squared L2 and $k = 10$. Recall is the mean fraction of ground-truth top-10 identifiers recovered, using only the first ten ground-truth entries even when a file stores a wider row. The refinement factor determines $c = \max\{k, \lceil \alpha k \rceil\}$ survivors. Throughout this chapter g_{LUT} denotes Cartesian table grouping and g_{scan} denotes parallel scan groups; the two parameters perform different work.

The chapter's analysis script reads existing measurements without running new GPU experiments. It verifies record counts and correctness fields, recomputes paired ratios and matched-recall values, and writes the numeric tables directly from those records. A companion source ledger records SHA-256 hashes of consumed files. Its source-checkout identifier records the audited repository snapshot; it does not imply that every historical binary was built from that commit. Raw run headers and family-specific reports retain the individual build provenance. This separation makes the current figures reproducible without relabelling historical runs.[1]

7.2 Modern GPU ANN Recall–QPS Frontiers

This section answers RQ4 by locating common-recall operating regions, including regions in which competing methods are faster.

7.2.1 Comparison scope and frontier construction

Figure 7.1 compares JHQ-GPU with CAGRA fp32, scalar-quantized CAGRA int8, IVF-RaBitQ, IVF-PQ and GPU brute force. CAGRA represents graph search; PQ and RaBitQ represent compressed inverted-file alternatives [5, 8, 9]. The four datasets are Vogue (932,328 × 768), arXiv (2,253,000 × 768), and the 999,000-vector OpenAI datasets at 1,536 and 3,072 dimensions. Each uses 1,000 queries and the same ground-truth file across methods.

Each method contributes its non-dominated measured operating points. An operating point is removed if another configuration of that method achieves at least its recall and throughput, with a strict improvement in one. Thus a frontier combines search parameters and sometimes distinct index configurations; it is not the trajectory of one fixed index. JHQ contributes 27–39 points per dataset, compared with 54–264

[1] The report repository provides `scripts/build_ch7.py`, `data/ch7/audit.json`, `data/ch7/matched.csv` and the source ledger `data/ch7/sources.csv`. The script requires the sibling `JHQ_GPU` checkout containing the archived measurements.

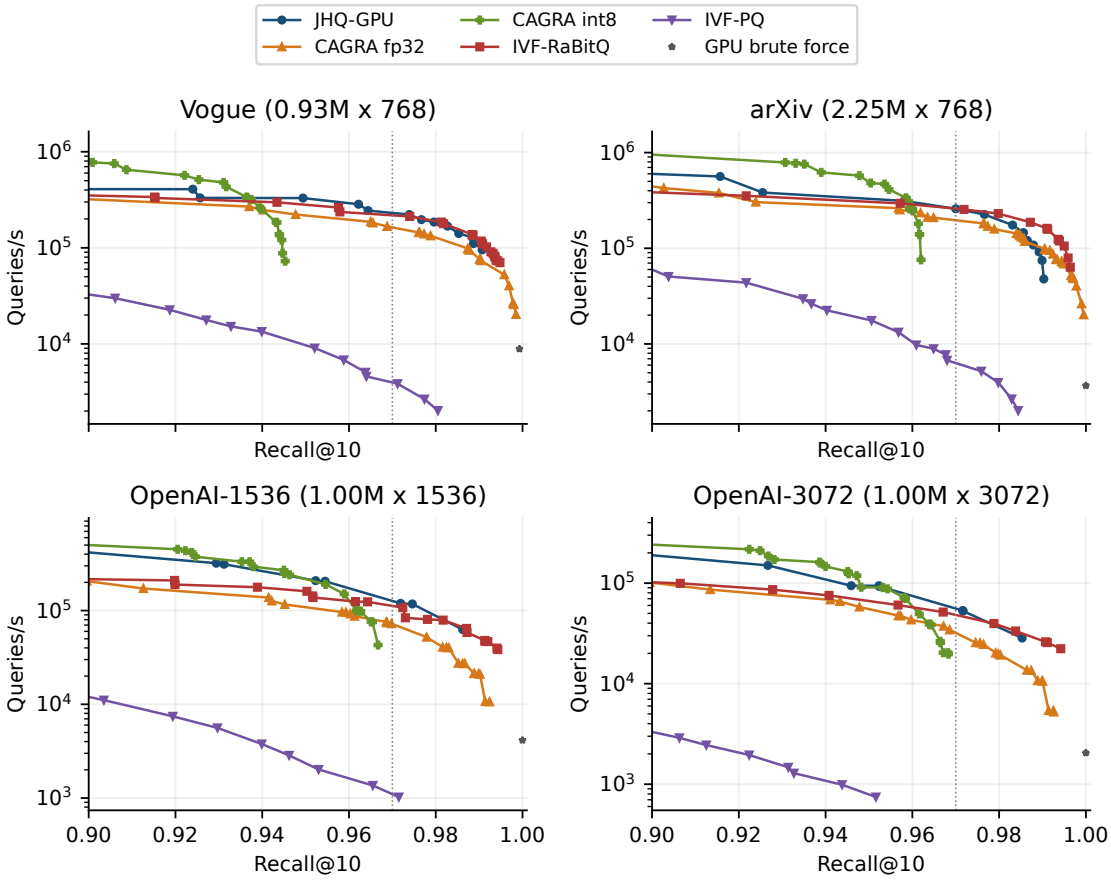


Figure 7.1: Measured host-to-host recall–QPS frontiers on the four E1 datasets. The displayed recall window begins at 0.90, with a vertical line at 0.97; all baseline families, including CAGRA int8, participate in frontier construction. Lines connect measured points; throughput uses a logarithmic axis.

for the baseline families. The E1 parser finds 1,590 valid points, no failure records and no invalid-identifier or duplicate-row flags. This count establishes the size of the search sweep, not 1,590 independent replications.

The parameter coverage is substantive. CAGRA varies graph degree 32/64, search width 1–8 and intermediate top-k capacity 32–512. RaBitQ varies 4/8/9-bit budgets, supported search modes and probe counts up to 1,024. PQ varies code length, lookup precision and probe count. JHQ varies both probe count and refinement budget, extending its initial grid where recall coverage is insufficient. The resulting comparison is conditional on these recorded grids. A curve ending before a target recall indicates an unmeasured operating region, not a proof that the method cannot reach it.

These curves compare achievable throughput, not equal memory consumption. JHQ stores approximately $d/8$ primary bytes and d residual bytes per vector, plus

Table 7.2: Recall 0.97 comparison. Each ratio is interpolated / measured; values above one favour JHQ. JHQ kQPS is the interpolated estimate. The small arXiv–RaBitQ difference changes direction with the estimator.

Dataset	JHQ kQPS	vs CAGRA fp32	vs IVF-RaBitQ
Vogue	230.8	1.416 / 1.537	1.059 / 1.046
arXiv	258.6	1.317 / 1.422	0.996 / 1.015
OpenAI-1536	126.7	1.747 / 2.294	1.130 / 1.111
OpenAI-3072	55.9	1.747 / 2.088	1.157 / 1.346

metadata. RaBitQ at nine bits has a comparable nominal $9d/8$ code budget, but its 4/8-bit configurations use less storage. CAGRA int8 also stores graph edges; PQ code lengths vary; CAGRA fp32 retains FP32 vectors. Taking the envelope across these choices is useful for deployment decisions, but cannot isolate a representation advantage at a fixed memory budget. The memory evidence is discussed separately in Section 7.7.

7.2.2 Matched recall and opposing operating regions

Table 7.2 quantifies the region around recall 0.97 using two estimators. The interpolated estimator is linear in log throughput between adjacent frontier points and never extrapolates beyond coverage. The measured estimator selects the fastest sampled point reaching the target. The interpolated estimator treats a measured recall within 5×10^{-4} of the target as a match; the measured estimator requires recall at least equal to the target. Interpolation reduces sensitivity to unequal grid spacing; the measured estimator corresponds to an actually recorded configuration. Neither creates a new measurement at exactly the requested recall.

Against CAGRA fp32, interpolated ratios at this target range from 1.317× on arXiv to 1.747× on the two OpenAI datasets. Against IVF-RaBitQ, the corresponding values are 1.059×, 0.996×, 1.130× and 1.157×. The arXiv result is a near tie: its interpolated rate is slightly lower, while the measured threshold comparison is slightly higher. Reporting this as an unequivocal win would give the interpolation convention more evidential weight than the measurement supports.

Across targets 0.90–0.985, JHQ's interpolated advantage over CAGRA fp32 is approximately 1.10–2.44× in the covered comparisons. However, adding CAGRA int8 changes the low-recall interpretation: around 0.90–0.92, its lower-precision graph search can be faster than JHQ. At higher recall, IVF-RaBitQ often regains the lead, including several targets at 0.98 or above. On arXiv around 0.99, CAGRA fp32 is

also faster than JHQ in the recorded grid. The defensible claim is therefore that JHQ has useful competitive regions, particularly around moderately high recall, rather than that it dominates modern GPU ANN methods.

CAGRA int8 reaches maximum measured recalls of 0.9453, 0.9620, 0.9667 and 0.9683 on Vogue, arXiv, OpenAI-1536 and OpenAI-3072, respectively. It therefore has no qualifying point at 0.97 in these grids. At targets 0.90–0.92 it is approximately 1.20–1.98× faster than JHQ by the same interpolation rule; it remains faster at 0.95 on arXiv and at 0.94 on both OpenAI datasets. The 0.97 result concerns methods with measured coverage and does not extend to every target below 0.97.

The two estimators can disagree substantially where the grid is sparse. For example, at recall 0.97 the OpenAI-1536 comparison with CAGRA fp32 is 1.747× by interpolation but 2.294× using qualifying measured points. The latter includes the cost of whichever baseline point next exceeds the threshold. It should not be presented as a directly measured same-recall advantage of that magnitude. The companion matched table preserves both estimators and marks targets without coverage, enabling readers to distinguish an operating-region result from a sampling artefact.

7.2.3 Interpretation and comparison limits

JHQ combines inexpensive primary screening with selective residual access. This provides a plausible explanation for competitive throughput when a small fraction of scanned candidates needs refinement. The kernel and budget ablations below test parts of that explanation directly. The cross-system curves alone cannot assign the gain to any one component: they also differ in routing, representation, preprocessing, index building and software interfaces. In particular, increasing dimension across these datasets is accompanied by changes in data distribution and dataset size. The stronger OpenAI ratios do not constitute a controlled demonstration that dimensionality causes JHQ's advantage.

Residual comparison limits remain after aligning the main timing boundary. JHQ is called from C++, whereas CAGRA and PQ use Python bindings; small host overhead differences remain even with reused resources. JHQ's output copy targets pageable host memory, whereas baseline outputs use pinned memory. Compilation targets differ, and unrelated host load is not fully controlled. These effects are relevant to small margins. Five rounds characterize repeated execution of the built indexes;

they do not estimate variation across independent index builds or independent machines.

7.3 Cartesian LUT, Packed Loads and Lane Refinement

For RQ2, the ablations separate representation-preserving table and load changes from cooperative residual scoring and measure their application effect as well as their kernel effect.

7.3.1 Exact table factorization and its throughput consequence

Cartesian factorization changes the evaluation of the represented primary distance, rather than training a new quantizer. Splitting an eight-bit Cartesian code into two four-bit components replaces 256 table entries with $2 \times 16 = 32$. With FP32 entries, this changes 1,024 bytes to 128 bytes per corresponding table, an eightfold reduction. The mathematical score is preserved; floating-point evaluation order can still affect the final bits. Smaller tables alone therefore do not justify calling the complete ANN output bit-identical.

Figure 7.2 compares Cartesian table grouping on six datasets, probe counts 8/32/128/512, fixed $\alpha = 100$, and $g_{\mathrm{LUT}} \in \{1, 2, 4, 8\}$. Each arm has five timing rounds; batch size is 1,000 except Stella's 999 queries. Two groups have the highest median throughput in 22 of 24 cells. The exceptions are arXiv at 128 probes, where one group is about 0.23% faster, and OpenAI-3072 at eight probes, where four groups are about 0.84% faster. These narrow differences are best treated as near ties. The newer count is 22/24, rather than the historical ADC paper's 23/24.

The two-group/one-group ratio ranges from 0.998 to 1.914. Four and eight groups each require only 16 total entries, but differ in the number and granularity of lookups. Their performance provides a useful control: reducing table storage beyond the two-group design does not necessarily improve search. Additional indexing and accumulation work can outweigh storage savings. The measurements establish the net trade-off; without a separate occupancy or instruction-counter experiment, they do not identify one hardware effect as the sole cause. The maximum reported recall spread within any four-arm cell is 10^{-4}.

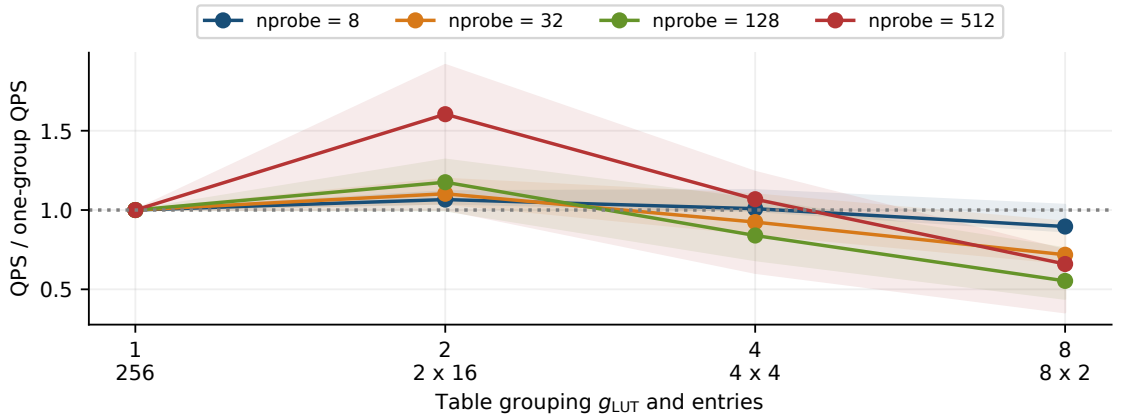


Figure 7.2: Cartesian table grouping at fixed $\alpha = 100$ (v71). Group counts describe g_{LUT}, not parallel scan groups. Curves are medians of per-dataset QPS ratios to the one-group arm; shading shows their range.

7.3.2 Packed access and cooperative residual scoring

The historical packing experiment compares byte-oriented and word-oriented access while holding the dataset, probe count and table grouping fixed. Across six datasets, four probe counts and $g_{\mathrm{LUT}} \in \{1, 2\}$, the 48 pairs yield throughput ratios from 1.012 to 1.658, with maximum recall difference 10^{-4}. Gains are generally smaller at eight probes and larger at 512. Both alternatives already use a subspace-major layout; this experiment supports packed access, not a claim that a physical transpose alone produced the improvement. Each arm has one reported aggregate rate, so this older evidence has weaker uncertainty information than the five-round v71 sweep.

Figure 7.3 and table 7.3 then separate wider loads from lane cooperation in residual refinement. L0 is the original scalar path; L1 changes the load path without distributing a candidate across multiple lanes; L16 assigns 16 lanes to a candidate. The same process rotates arm order across five timing rounds, with three search calls per arm per round. For the twelve ordinary configurations, L16 improves end-to-end throughput by 1.015–1.789×. An additional arXiv configuration at $\alpha = 200$, explicitly a high-budget control, reaches 2.383×; it is not the gain range of the ordinary selected budgets.

The load-only control is bit-identical to L0 in the tested cases, yet already improves arXiv throughput by 1.368×. Lane cooperation adds a larger reduction in refinement time. On arXiv at $\alpha = 100$, the profiled refinement kernel falls from 4,277.5 to 716.8 μs, a 5.97× gain, while total throughput improves 1.789×. OpenAI-3072's refinement kernel improves 8.81×, but total throughput improves 1.171×. This is

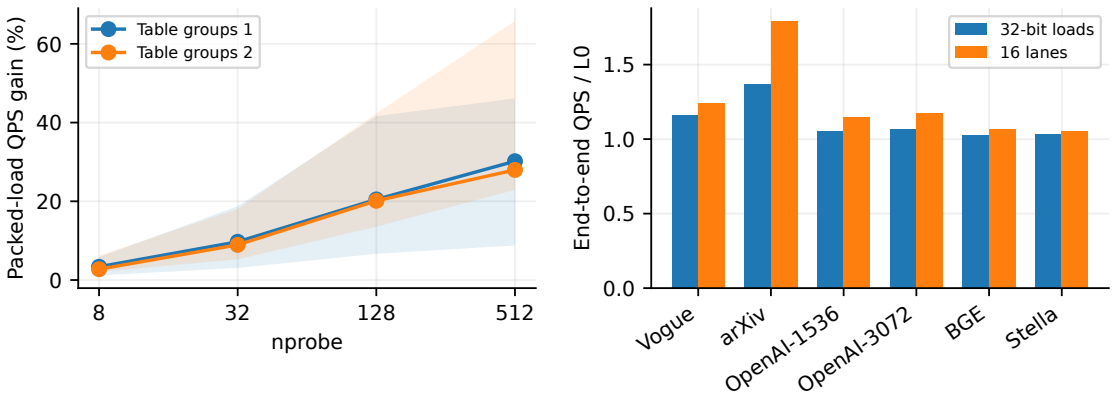


Figure 7.3: Packing and refinement controls. Left: median packed-load QPS gain across six datasets, with shaded minimum–maximum ranges, for each table grouping and probe count. Right: measured load-only and 16-lane end-to-end gains at 128 probes and the ordinary dataset-specific budgets. These panels come from different experiment families.

Table 7.3: Refinement at 128 probes. Ratios are paired end-to-end gains over L0, except the last column, which is the profiled refinement-kernel gain. The kernel and application gains must not be interchanged.

Dataset	α	$L1/L0$ QPS	$L16/L0$ QPS	Kernel gain
arXiv	100	1.368	1.789	5.97
BGE	32	1.026	1.062	1.97
OpenAI-1536	8	1.055	1.145	3.34
OpenAI-3072	4	1.063	1.171	8.81
Stella	16	1.030	1.053	2.66
Vogue	64	1.161	1.240	3.72

consistent with Amdahl's law: acceleration of a small stage cannot remove the cost of routing, screening, final selection or transfers. It also motivates the subsequent scan and probe experiments, which target the remaining work.

A quantitative Amdahl check uses the refinement kernel's baseline time as a fraction f of host-to-host baseline latency and its kernel speedup s, giving $1/(1 - f + f/s)$ if the remaining time is unchanged. The fractions are approximately 0.529 on arXiv and 0.168 on OpenAI-3072, producing estimates of 1.787× and 1.175×, close to the measured 1.789× and 1.171×. These combine separate kernel-profile and wall-time measurements and are explanatory estimates.

Cooperative reductions alter floating-point addition order. The tested multi-lane variants have very small distance differences, with maximum reported relative error about 1.6×10^{-6}, and occasional changes near ranking boundaries. The evidence therefore supports numerically close scoring with measured end-to-end gains, rather than universal bitwise equivalence. Sanitizer checks cover the recorded refinement

and top-k kernels on small query subsets; they are useful implementation checks, not a proof for arbitrary inputs.

7.4 Global Top-c, Shared Bound and Probe Correctness/Ablation

This section answers the correctness part of RQ2 before examining the cost of grouped scanning, threshold sharing and radix probe selection.

7.4.1 Global primary selection before performance

The grouped-scan experiment addresses a correctness requirement that cannot be evaluated through recall alone. Dividing a query's candidates among blocks must preserve the global primary top-c; assigning each block only c/g_{scan} survivors can discard a globally competitive candidate from a disproportionately productive group. JHQ's grouped-scan path instead retains up to c candidates in every group and merges the local lists. Selection uses the complete key $(D_{\text{P}}, \text{position})$, where position breaks primary-distance ties deterministically.

The local retention rule has a direct justification. A candidate in the global top-c has fewer than c preceding keys in the entire candidate set, and hence fewer than c in its own group. It must therefore appear in that group's local top-c. Taking the top-c of the union, with the same total order, recovers the global top-c. This statement concerns primary scores over the visited candidates. It does not make IVF routing exact, turn primary scores into raw distances, or make the final ANN answer an exact nearest-neighbour result.

The grouped-scan checker covers four workloads, batches 8/32/256/1,000 and group counts 1/2/4/8, giving 64 comparisons. It reports no non-tie primary selection differences from the original one-block path, zero change in compared primary distances, and bit-identical final top-10 identifiers and distances for every used query. The original path sorts on distance alone, so its primary cutoff can select a different member of a tie; these differences do not violate the new complete-key specification. They also did not change final outputs in this particular experiment.

The number of used queries deserves attention. The harness submits only complete batches: out of a 1,000-query pool it uses 992 queries for batch 32 and 768 for batch 256, whereas batch 8 and batch 1,000 use all 1,000. Correctness counts and timing populations therefore depend on the batch regime. They cannot be described as the

identical 1,000-query test in every row. Recorded memcheck, racecheck and synccheck runs provide additional checks on smaller subsets and on the named scan/merge kernels.

7.4.2 Shared bounds preserve the selection target

Keeping a full local top-c is safe but potentially expensive. With the shared-bound component (v63), groups of one query publish a shared bound using the same complete key. Once any group has established c keys no greater than a bound, a candidate worse than that bound cannot enter the global top-c. A stale, looser bound can cause extra admissions, but cannot legitimately exclude a better candidate. This requires that a published bound be backed by enough valid candidates and that atomic updates preserve the complete-key ordering; an approximate distance threshold alone would not establish the same tie behaviour.

The implementation still computes every scanned candidate's complete primary score before deciding admission. The bound reduces selection, compaction, sorting and downstream merging work; it does not prune primary distance evaluation or avoid reading every rejected primary code. This scope matters when interpreting an apparent scan-stage speedup. A faster scan stage need not mean fewer evaluated distances.

The shared-bound checks compare shared-bound groups 2/4/8 with their independent-bound counterparts across the four workloads and four batch sizes. All 48 comparisons report identical primary top-c and final outputs. Five-round paired throughput measurements range from about 0.994 to 1.562× relative to independent bounds. Thus sharing substantially helps some expensive cases but is not uniformly faster. On arXiv at $\alpha = 200$, batch 1,000 and eight groups, the profiled merge cost falls from approximately 2,629 to 471 μs, while scan falls from 7,968 to 5,513 μs in the same paired profile. The result supports reducing excess intermediate work rather than claiming that more scan groups are always beneficial.

A rejected alternative was to allocate smaller static local quotas. The campaign's distribution measurements place 46–81% of the global survivors in the first candidate group for the pilot workloads. Equal quotas would frequently require a correctness fallback, and calibrated unequal quotas still leave the dominant group on the critical path. These observations motivated the exact shared bound. They are pilot measurements of those workloads, not a theorem that every possible quota policy is inferior.

Table 7.4: v69/v65 end-to-end throughput ratios for the exact FP32 probe change. Each workload–batch comparison has ten timing rounds per binary, collected in alternating runs. Values above one favour v69.

Dataset	Batch 8	32	256	1,000
arXiv	1.074	1.063	1.035	1.034
Vogue	1.109	1.103	1.052	1.041
OpenAI-3072	1.167	1.117	1.055	1.050

7.4.3 Exact probe selection and the limits of pipeline identity

Once screening becomes faster, IVF probe selection becomes a larger fraction of small-batch time. The radix probe primitive (v69) changes how the smallest centroid scores are selected while retaining FP32 scoring. Its synthetic checker covers 1,792 cases and reports zero failures. Checks of the real arXiv, Vogue and OpenAI-3072 workloads find identical probe identifiers and identical primary top-c positions and distances for all 1,000 queries. These are more specific observations than a single unchanged recall number: they test the routing and screening outputs directly.

Table 7.4 shows end-to-end improvements of 1.034–1.167×. At batch 8, the profiled probe stage falls from 56.5 to 22.6 μs on arXiv, 52.7 to 14.7 μs on Vogue, and 77.8 to 16.7 μs on OpenAI-3072. Smaller full-batch gains are expected because the unmodified stages account for more of total elapsed time. These are within-family ratios; they should not be multiplied by the preceding v62 or refinement gains as if all measurements shared a single baseline.

Vogue isolates an important distinction between the probe primitive and cross-process pipeline reproducibility. The probe/primary dump is byte-identical across the two binaries for all 1,000 queries in the one-group, batch-8 test, yet final recall is 0.9642 versus 0.9644. The same baseline binary also changes from 0.9642 in the identity run to 0.9644 in each of its two timing runs. Thus the observed variation does not require the probe change; in the dump-controlled case it arises after the recorded primary-selection boundary. Residual representations, inputs and score computation remain possible sources, whose root cause is unconfirmed. Within each checked process, scheduled and one-block outputs remain bit-identical. Appendix A records the dump coverage so that it is not mistaken for every timed path.

Finally, the optional v70 FP16 experiment is a separate approximation. Its probe sets match FP32 exactly on 936/1,000 arXiv queries, 979/1,000 Vogue queries and 998/1,000 OpenAI-3072 queries, with very high mean set overlap. High overlap and

small recall changes do not make the primitive exact. Those results are excluded from the main E1 frontier and from the exact-probe claim.

7.5 Held-out Budget Quality, Throughput and Payback

RQ3 concerns quality and amortized cost against credible fixed budgets. Accordingly, the comparison includes both the inherited $\alpha = 100$ and the stronger universal fixed control $\alpha = 64$.

7.5.1 Unlabelled selection and a labelled diagnostic oracle

The budget experiment fixes one trained index per dataset–probe cell and compares fixed budgets, sampled calibration, full-pool calibration, same-sample enumeration and an offline oracle. The six datasets are tested at 128 and 512 probes. Even-indexed queries form a calibration pool of 500; odd-indexed queries form a disjoint held-out pool of 500, except Stella, which has 499. Samples of $S \in \{32, 64, 128\}$ are drawn from the calibration pool without replacement within each draw, using a fixed random seed, with 64 draws per size. Repeated draws reuse a fixed finite query pool; they are not 64 independent samples of future workload drift.

The descending grid is $A = (200, 100, 64, 32, 16, 8, 4, 2)$. Calibration compares candidate top-10 identifier sets with the $\alpha_{\max} = 200$ reference. For sample $\mathcal{Q}_S$, the acceptance statistic is

$$D_S(\alpha) = \sum_{q \in \mathcal{Q}_S} \left(10 - |\mathcal{R}_\alpha(q) \cap \mathcal{R}_{200}(q)|\right), \qquad D_S(\alpha) \leq 1. \tag{7.1}$$

The allowance is one disagreeing result slot over the entire sample, not one per query and not a ground-truth recall tolerance. No labels are supplied to the deployed rule. The reference is itself approximate, so perfect agreement can reproduce its errors.

The offline oracle has a different role. After observing held-out recall for all eight fixed budgets, it selects the fastest measured arm within 10^{-3} of the best measured held-out recall. This is a diagnostic benchmark, not an implementable label-free decision. Likewise, "full pool" means applying the same calibration search to all 500 calibration queries; it does not mean enumerating every budget with held-out labels. The distinctions prevent an apparent oracle match from being mistaken for a quality guarantee.

Table 7.5: Held-out quality at $S = 128$. Rule α is the median across 64 draws; oracle α uses held-out labels. $\overline{R}_{\text{rule}}$ is reconstructed mean recall. "Losses" counts draws with recall loss $> 10^{-3}$ relative to the best tested held-out recall; L_{95} is the logged lower-order 95th-percentile loss estimate. R_{100} is fixed-budget recall.

Dataset	Probe	α: rule/oracle	R_{100}	$\overline{R}_{\text{rule}}$	Losses	L_{95}
Vogue	128	64 / 64	0.9680	0.9677	5/64	0.0022
arXiv	128	100 / 200	0.9748	0.9751	40/64	0.0024
OpenAI-1536	128	8 / 8	0.9280	0.9279	4/64	0.0000
OpenAI-3072	128	4 / 4	0.9252	0.9252	0/64	0.0000
BGE	128	32 / 32	0.9198	0.9190	1/64	0.0008
Stella	128	16 / 16	0.9892	0.9884	6/64	0.0030
Vogue	512	64 / 64	0.9916	0.9915	2/64	0.0002
arXiv	512	200 / 200	0.9878	0.9885	32/64	0.0016
OpenAI-1536	512	8 / 8	0.9740	0.9738	4/64	0.0000
OpenAI-3072	512	4 / 8	0.9704	0.9702	0/64	0.0002
BGE	512	64 / 64	0.9670	0.9661	24/64	0.0014
Stella	512	16 / 16	0.9954	0.9945	7/64	0.0030

7.5.2 Budget quality and sampling risk

Table 7.5 reports the $S = 128$ result. The median selected budget matches the oracle budget in ten of twelve cells; it differs on arXiv at 128 probes and OpenAI-3072 at 512. The selected budgets span 4–200, indicating substantial workload dependence. However, a median budget and a mean recall describe different aspects of the draw population. They need not correspond to the performance of one arm. In addition, mean recall in the table is reconstructed from the printed best recall minus printed mean loss, so its four-decimal precision is approximate.

The important counterexample is arXiv. At 128 probes, 40/64 draws lose more than 10^{-3} recall even though mean loss is only 0.0009; at 512 probes, 32/64 draws cross the threshold. BGE at 512 probes has 24/64 threshold crossings and a 95th-percentile loss of 0.0014. Stella has fewer crossings but a 95th-percentile loss of 0.0030 at both probe depths. Thus mean quality, violation frequency and tail magnitude must be read together. OpenAI-3072 has no observed threshold crossings in these draws, which supports a favourable result on this pool, not a zero-risk statement about unseen queries.

The percentile convention also matters for this small sample of draws. The harness reports the sorted loss at zero-based index $\lfloor 0.95(64-1) \rfloor = 59$, rather than interpolating between order statistics. Consequently, four nonzero tail losses can coexist with a printed $L_{95} = 0$, as on OpenAI-1536. The table preserves this logged

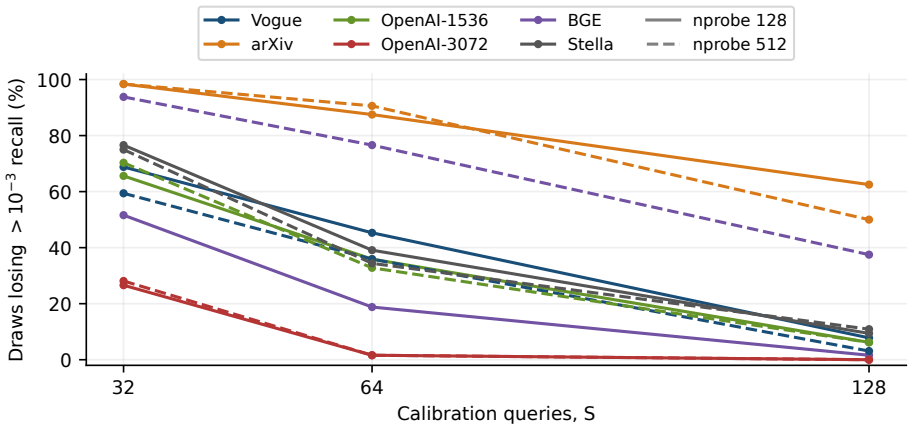


Figure 7.4: Held-out risk across sample sizes. The vertical axis is the percentage of calibration draws losing more than 10^{-3} held-out recall relative to the best tested budget. Increasing S also tightens the allowed slot-disagreement fraction because equation (7.1) keeps threshold one.

statistic and the exact crossing count; zero in that column does not mean every draw has zero loss.

Figure 7.4 shows the effect of sample size. A larger sample usually makes selection more conservative, but also tightens the normalized allowance $1/(10S)$. Same-sample enumeration agrees with bisection in all 2,304 draws ($12 \times 3 \times 64$), while taking 1.75–2.01 times as long in the printed logs.

The measured default path performs exact primary top-c, and final top-k scans all c refined candidates with ID-based tie breaking. The lossy `K_LOCAL=4` retention path is an opt-in alternative selected by `JHQ_EXACT_TOPCK=0`; it is not the explanation for this budget experiment. Exact primary selection yields nested survivor sets under fixed scores and a common total tie order, making the sampled agreement criterion monotone under those conditions. The remaining qualifications are the distance-only cutoff ties of the one-block path and the batch-dependent choice between one-block and complete-key grouped scans. Small calibration batches and held-out batches can therefore use different execution paths. Enumeration agreement verifies the sampled decisions on this grid, not held-out recall or unconditional nesting across paths.

A fixed budget is also imperfect. Table 7.6 evaluates all eight fixed choices against the diagnostic quality target. Fixed $\alpha = 100$ qualifies in ten of twelve cells; $\alpha = 200$ qualifies in all twelve but can sacrifice substantial throughput. Full-pool calibration is not identical to the oracle either: on Vogue it selects 100 at both probe depths while the oracle selects 64. Agreement-based selection and fastest quality-constrained selection solve different problems, even with a larger calibration

Table 7.6: Fixed-budget controls over twelve held-out cells. Qualifying means within 10^{-3} of the best tested recall at printed precision. Worst recall loss uses all cells; QPS deficit uses only cells where that arm qualifies. A dash means no qualifying cells.

α	Qualifying cells	Worst recall loss	Max qualifying QPS deficit
2	0/12	0.3100	–
4	2/12	0.1774	2.3%
8	4/12	0.0930	2.1%
16	6/12	0.0408	4.4%
32	7/12	0.0170	9.1%
64	10/12	0.0050	26.6%
100	10/12	0.0016	40.9%
200	12/12	0.0000	57.1%

sample. Full-pool calibration costs 9.5–60.3 ms. In these twelve cells $\alpha = 200$ attains the best measured recall and therefore serves as a high-budget coverage control; its 12/12 qualification is not an independent quality guarantee. The grid's throughput is not strictly monotone: for example, OpenAI-3072 at 128 probes gives 128.7 kQPS at $\alpha = 2$ and 145.0 at $\alpha = 4$. Three timed calls do not distinguish timing variation from a configuration-dependent execution cost, so no mechanism is assigned to that inversion.

7.5.3 Estimated throughput and calibration economics

Unlike E1, this family times each fixed arm with a warm-up and three search calls. The rule's reported QPS is then the arithmetic mean of the already measured QPS values of its 64 selected arms. It is not a new timing of a service repeatedly calibrating and executing the policy. At $S = 128$, this selected-arm estimate improves over fixed $\alpha = 100$ in ten cells, by 1.028–1.691×. On arXiv the ratios are 0.908 and 0.917: mean recall improves, but throughput falls. Those cells have no throughput payback.

The stronger $\alpha = 64$ comparison reduces the aggregate gain: mean selected-arm QPS wins in 9/12 cells, with geometric-mean ratio 1.054× and range 0.821–1.367×. It is nearly unchanged on Vogue and improves only 0.6% on BGE at 512 probes; OpenAI gains range from 1.052 to 1.367×. Both fixed 64 and fixed 100 qualify in 10/12 cells, failing the two arXiv cells. The useful property of calibration is matching heterogeneous refinement demands, rather than uniformly beating an already sensible fixed budget.

ArXiv's threshold crossings are relative to the best tested held-out recall, not relative to the default configuration. Fixed 100 already loses 0.0012 and 0.0016

Table 7.7: Calibration economics, $S = 128$. Rates are kQPS and rule rate is mean selected-arm QPS. Ratios compare with fixed 100 and fixed 64. Payback in the last column is relative to fixed 100 using 500 queries, or 499 for Stella; "Never" denotes no throughput payback.

Dataset	Probe	Q_{100}	$\overline{Q}_{\text{rule}}$	vs 100	vs 64	Cal. ms	Batches
Vogue	128	202.9	224.4	1.106	1.004	4.0	17.0
arXiv	128	207.9	188.8	0.908	0.821	4.2	Never
OpenAI-1536	128	167.5	268.5	1.603	1.367	3.9	3.5
OpenAI-3072	128	85.7	145.0	1.691	1.230	5.7	2.4
BGE	128	116.2	131.2	1.130	1.051	6.5	13.2
Stella	128	95.7	110.3	1.153	1.087	7.5	10.8
Vogue	512	100.1	102.9	1.028	0.996	7.5	55.4
arXiv	512	83.1	76.2	0.917	0.870	9.7	Never
OpenAI-1536	512	86.6	115.0	1.327	1.268	8.8	6.2
OpenAI-3072	512	42.8	52.1	1.219	1.052	13.1	6.2
BGE	512	49.2	51.1	1.039	1.006	15.8	41.2
Stella	512	34.6	36.0	1.041	1.028	21.3	37.1

recall at probes 128 and 512, while the sampled mean loss is 0.0009 in both cells. Mean quality therefore improves over the default despite some draws exceeding the oracle-relative target.

Calibration costs 3.9–21.3 ms in these logs. This includes reference search and agreement checks, with any included search setup inside the measured calls, but excludes gathering sample vectors into the input buffer. For an equal-sized subsequent batch, define

$$B^* = \frac{T_{\text{cal}}}{T_{100} - T_{\text{rule}}}, \qquad T_{\text{rule}} < T_{100}. \tag{7.2}$$

Using $T_{\text{rule}} \approx n/\overline{Q}_{\text{rule}}$ gives the estimates in table 7.7: 2.4–55.4 subsequent held-out-sized batches against fixed 100 in favourable cells. Against fixed 64, the corresponding range is 5.7–404.3 batches in nine favourable cells; Vogue at 512 probes has no throughput payback. Figure 7.5 uses the actual held-out query counts for both controls.

The calculation is optimistic. In general, $1/\mathbb{E}[Q] \leq \mathbb{E}[1/Q]$, so inverting mean QPS understates mean execution time when calibration draws choose arms with different rates. Sample gathering, future query drift and recalibration overhead would further affect amortization. A policy reused for a long stationary workload can benefit; a short job or a changing query population may not. The present experiment measures neither drift detection nor a production recalibration schedule.

For historical comparison, the earlier scalar-refinement implementation (v57) reported 1.054–2.773× gains over fixed 100 in 10/12 cells and 5.2–30.8 ms calibration. The newer lane-refined pipeline's smaller gains are consistent with less removable

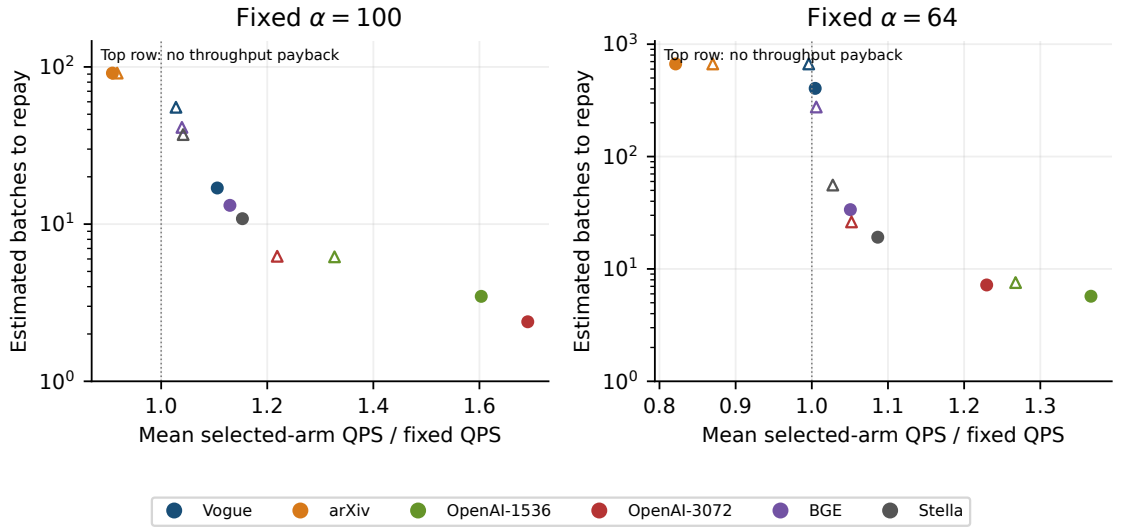


Figure 7.5: Estimated calibration payback relative to fixed budgets 100 and 64. Filled circles denote 128 probes and open triangles 512 probes. Top-row points have no throughput payback. Estimates invert mean selected-arm QPS and use 500 queries, or 499 for Stella.

refinement time. The runs share the budget protocol and grid, but this cross-campaign comparison does not isolate refinement as the sole cause of the change.

7.6 Batch Regimes, G Scheduling and Workload Skew

This section answers the performance-regime part of RQ2: when additional scan groups supply useful parallelism, and when their extra work dominates.

7.6.1 Parallelism versus repeated selection work

The scan experiments reveal two execution regimes. With one block per query, batch 8 exposes only eight blocks on a 170-SM device. Candidate partitioning increases the available block parallelism. In the independent-bound grouped-scan experiment, eight groups improve end-to-end throughput by 2.06–2.76× at batch 8. Profiling supports a stage-level explanation: arXiv's scan falls from about 849 to 216 μs, Vogue's from 1,059 to 246 μs, and OpenAI-3072's from 728 to 155 μs. The additional merges cost approximately 16.7, 9.1 and 3.7 μs, respectively. The net reduction is large even after paying for merging.

At batch 1,000 there are already enough independent queries to expose many blocks. Additional groups repeat table staging, local-buffer admission and sorting,

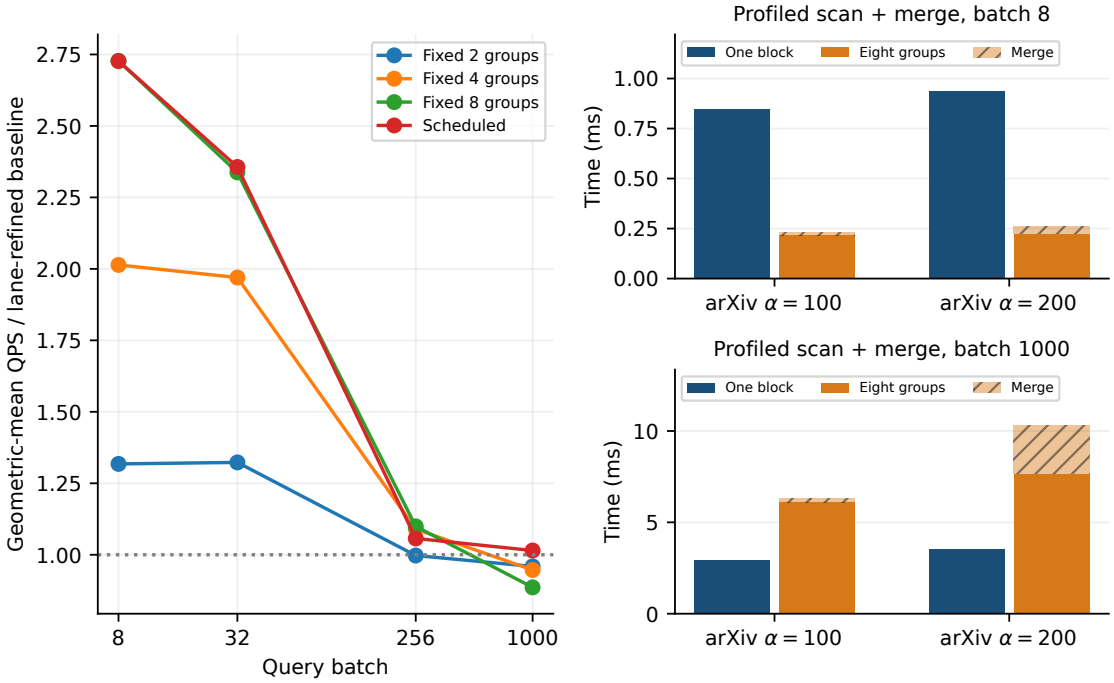


Figure 7.6: Execution regimes. Left: fixed shared-bound group counts and scheduled execution relative to the lane-refined one-block baseline (13 cells at small batches, 11 at batch 1,000). Right: paired scan and merge profiles at batches 8 and 1,000 for arXiv budgets 100 and 200. Hatching denotes merge time. The right panels use the independent-bound scan experiment; they explain a cost, not the scheduler's entire speedup.

and produce extra merge traffic. The independent-bound grouped path can fall to about 0.46× its one-block baseline on arXiv at $\alpha = 200$. The full-batch profiles show scan increasing from 2,922 to 6,079 μs at $\alpha = 100$ and from 3,530 to 7,652 μs at $\alpha = 200$, with an additional 2,624 μs merge in the latter case. This opposing result is central to the design: parallel decomposition solves insufficient concurrency, but incurs work that can dominate once concurrency is adequate. Shared bounds reduce that extra work but do not eliminate the regime change.

7.6.2 A fixed rule evaluated on further configurations

The integrated batch scheduler (v65) chooses a scan-group count using batch size, block size and a workload-skew estimate. Its fill rule relates the potential resident threads to a fixed 0.6 fraction of the device's thread capacity. Its skew rule requires at least four groups when the coefficient of variation of candidates per query is at least 0.3. The larger suggested group count is used, with the one-group case mapped to the original one-block scan. The skew statistic is estimated once from 512 database vectors used as pseudoqueries, not from ground-truth labels or the

Table 7.8: Integrated execution gains over v61 L16. Entries are geometric means of per-cell median paired speedups from five rounds. "Held out" excludes the four configurations used to set the rule; it means held-out configurations, not the budget experiment's held-out query split.

Batch	Cells	$g_s = 2$	$g_s = 4$	$g_s = 8$	Scheduled	Held out
8	13	1.318	2.014	2.728	2.727	2.930
32	13	1.323	1.970	2.337	2.356	2.560
256	13	0.997	1.090	1.099	1.057	1.041
1000	11	0.959	0.947	0.886	1.015	1.012

evaluated query pool. The constants were chosen on four workload configurations before the additional configurations were measured.

Table 7.8 recomputes the results from the raw timing arrays. The scheduled geometric-mean gains are 2.727×, 2.356×, 1.057× and 1.015× at batches 8, 32, 256 and 1,000. There are thirteen configurations at the first three batch sizes, including an additional high-budget arXiv case, but only eleven at batch 1,000: Stella's 999-query pool cannot provide a complete 1,000-query batch. The held-out subsets contain nine and seven configurations, respectively. A plot connecting these means summarizes regimes rather than a perfectly balanced repeated-measures population.

Fixed eight-group execution matches the scheduled mean closely at batch 8 but falls to 0.886× at batch 1,000. Scheduling largely retains the small-batch gain while avoiding that full-batch loss. It is nevertheless a heuristic, not an oracle. At batch 256 its worst ratio of best fixed throughput to scheduled throughput is 1.217, on a BGE configuration. Candidate-count CV does not capture every cost of list traversal, selection or merging. This failure identifies a limitation of the chosen feature rather than evidence that execution scheduling is unnecessary.

Vogue illustrates why batch size alone is insufficient. Its measured candidate-count CV is about 0.51, compared with roughly 0.08–0.11 for the original other workloads, and its heaviest query has around 2.2 times the mean candidate count. Splitting candidates can shorten a slow query that would otherwise determine completion of a synchronous batch. This explains why Vogue can still benefit from multiple groups at larger batches, although the CV observations are workload evidence rather than a general causal isolation of skew.

Finally, correctness is checked independently of timing. Across 250 variant/configuration comparisons and 234,320 query comparisons, the integrated campaign reports no non-tie primary-selection differences and no different final identifier sets. Final order can differ at ties; the entire campaign should not be

Table 7.9: Cold JHQ construction and separately measured resident memory. Train/add come from one recorded cold build per dataset. Memory comes from v71's two-group, 128-probe, $\alpha = 100$ runs and includes context and workspace. "Residual" is the modelled residual-code share of the measured used total, not the fraction read per query.

Dataset	Train s	Add s	Total s	Used GiB	Raw GiB	Residual
Vogue	1.06	0.27	1.33	1.47	2.67	45.4%
arXiv	0.59	0.52	1.10	2.58	6.45	62.5%
OpenAI-1536	0.64	0.38	1.01	2.39	5.72	59.7%
OpenAI-3072	1.34	0.65	1.99	4.14	11.43	69.0%
BGE	3.75	3.67	7.42	11.93	38.50	80.7%
Stella	3.74	6.67	10.40	20.44	67.81	82.9%

described as universally bit-identical. The throughput results concern sequential, synchronous submissions and do not establish tail latency under an open-loop arrival process.

7.7 Cold Construction, Resident Memory and Tested Scale

RQ1 is assessed through cold construction, resident memory and completed large-dataset searches; the E1 memory comparison also informs RQ4.

7.7.1 Cold builds and the lifecycle boundary

The principal search sweeps reuse trained-state caches. Loading that state and adding vectors is not a cold build. Separate logs explicitly train with an empty trained-state cache and then encode/add the database; table 7.9 reports their recorded training and add times. The resulting totals range from about 1.01 s for OpenAI-1536 to 10.40 s for Stella. Stella's 3.74 s training and 6.67 s add phases also illustrate that neither phase can be inferred reliably from the other or from dataset size alone.

Cold here refers to learned index state. The input loader can reuse memory-mapped FP32 data, and the reported train-plus-add interval does not include reading the original exported files from storage. The scheduler's pseudoquery calibration is performed after add and is outside that total. A deployment lifecycle would also account for data preparation, transfer where excluded by a given API, calibration, serialization and subsequent query time. The recorded totals are useful construction

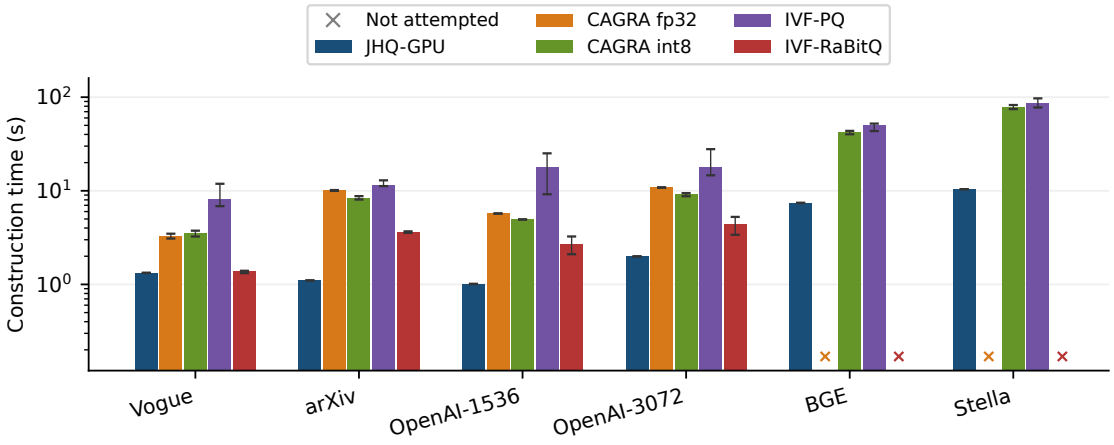


Figure 7.7: Recorded construction costs, with CAGRA int8 included. JHQ is one cold train-plus-add record; other bars are medians across successful measured index configurations, with minimum–maximum whiskers. Int8 includes dataset quantization; RaBitQ excludes the required serialization round trip. Missing large-dataset bars denote unattempted configurations, not measured build failures. The vertical axis is logarithmic.

measurements within their boundary, not a complete end-to-end ingestion service cost.

The comparison figure reports alternative construction configurations alongside these cold builds. Its method-specific ranges are not confidence intervals over equivalent repeated builds. Training samples, index parameters and representations differ, and JHQ has only one cold construction record per dataset. For RaBitQ, the recorded build function is separated from serialization/deserialization, although the benchmark requires that round trip before search. Omitting that extra step is appropriate only when clearly labelling the bar as build-function time; it does not measure time until the first usable search.

On Stella, successful isolated CAGRA int8 builds take approximately 74.55 and 82.45 s including dataset quantization, depending on graph degree; the measured IVF-PQ build median is 86.59 s across its three configurations. JHQ's 10.40 s record is substantially shorter under the reported boundaries, but does not establish a universally fastest builder or equal downstream recall. Construction speed, searchable quality and resident cost are separate deployment dimensions.

7.7.2 Resident allocation and where compression helps

Figure 7.8 decomposes resident memory. Its markers are device memory in use at the recorded measurement, not index-only allocations. The stacked components model primary and residual codes, correction factors and identifiers, centroids, per-batch

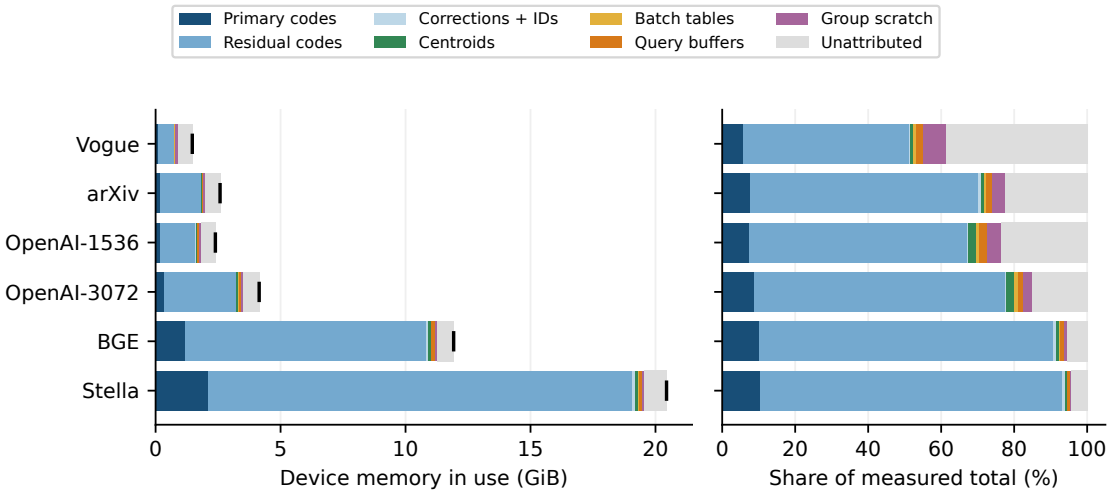


Figure 7.8: Resident-memory decomposition at two-group tables, 128 probes and $\alpha = 100$. Component sizes are modelled, markers are measured used totals, and hatching is the unattributed remainder. The totals include context and workspace; construction peaks are not measured.

tables, candidate buffers and list-group scratch. The remaining measured allocation stays explicitly unattributed; it may include runtime context, allocator effects and allocations omitted by the model. Assigning that remainder to residual codes would overstate the representation cost.

Residual codes dominate the large datasets: approximately 80.7% of BGE's 11.93 GiB and 82.9% of Stella's 20.44 GiB used totals. The corresponding raw FP32 vector arrays alone would require 38.50 and 67.81 GiB. JHQ therefore enables the tested resident search on one card by retaining a compressed hierarchy. Lowering α reduces residual codes accessed and scored per query, but leaves the stored residual array allocated. Similarly, Cartesian LUT factorization reduces query-specific table storage, not the database's residual-code precision.

Workspace also matters. The reported list-group scratch is approximately 93.8 MiB in these $\alpha = 100$ measurements, reserved for a batch capacity of 1,024 and up to eight groups even when the selected execution uses the one-block path. A smaller actual group count therefore does not imply that the allocation shrank proportionally. Conversely, these resident samples do not capture temporary construction peaks. The baseline harness also initializes some search memory pools after its index-memory measurement, so an after-build index delta cannot be used as a complete peak process-memory budget across methods.

The E1 measurements provide a separate cross-system view in table 7.10. Subtracting the baseline-measured 507.2 MiB context from JHQ's process total gives

Table 7.10: E1 allocation ranges. JHQ is process total minus a 507.2 MiB context estimate, including query workspace; CAGRA fp32 is index allocation measured during build. Ratio ranges combine the extrema for each dataset.

Dataset	JHQ net process GiB	CAGRA fp32 index GiB	Ratio range
Vogue	0.90–1.01	2.89–2.91	0.31–0.35
arXiv	2.01–2.19	6.84–6.98	0.29–0.32
OpenAI-1536	1.79–2.33	5.95–5.97	0.30–0.39
OpenAI-3072	3.54–3.82	11.67–11.68	0.30–0.33

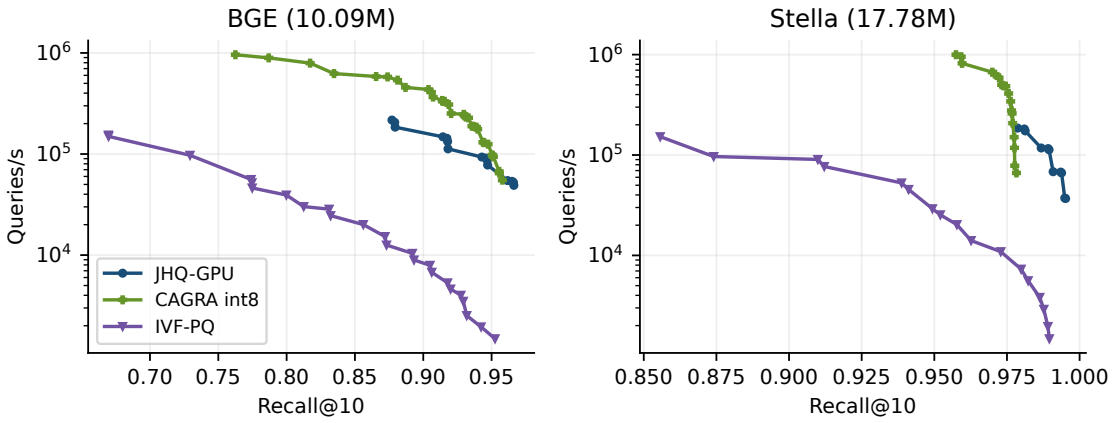


Figure 7.9: Large-dataset host-to-host frontiers, including the successful isolated CAGRA int8 reruns. BGE uses 1,000 queries and Stella 999. An endpoint is the end of the tested grid, not an intrinsic recall limit.

approximately 0.29–0.39 of CAGRA fp32's index allocation. JHQ's figure includes workspace, whereas CAGRA's is the build-time index delta. These are parameter-sweep ranges, not memory-matched recall points or a comparison of equal peak budgets.

7.7.3 Large-dataset feasibility and competitive regions

BGE contains 10,091,524 vectors and Stella 17,776,615, both with 1,024 dimensions and 32,768 IVF lists. Stella's queries are sampled from the corpus; its result therefore characterizes that benchmark rather than an independently collected external-query distribution. Figure 7.9 includes successful CAGRA int8 and IVF-PQ measurements. Full-precision CAGRA, RaBitQ configurations and FP32 brute force were not attempted in the large-scale comparison when their computed storage requirements exceeded the available device capacity. Those omissions are planned feasibility exclusions, not observed out-of-memory errors or measured recall failures.

Successful isolated reruns show that CAGRA int8 fits Stella in the tested configurations. On BGE, int8 is approximately 1.35–2.96× faster at the sampled common targets 0.88/0.90/0.92/0.94/0.95. At its measured maximum recall 0.9579, JHQ's interpolated QPS is 1.086× int8's. Thus the advantage changes direction near the high-recall end. The maximum tested recalls are 0.9579/0.9783 for int8, 0.9663/0.9952 for JHQ and 0.9525/0.9896 for PQ on BGE/Stella.

On Stella, one concrete high-end comparison is 185.4 kQPS at recall 0.978679 for JHQ versus 66.3 kQPS at 0.978278 for int8, a 2.80× rate ratio with slightly different recall. This is an explicit pair of measured points, not a uniform matched-recall gain: much of JHQ's higher-recall region has no int8 coverage, and JHQ's highest-recall points trade throughput for quality. Corrected-run exclusions and harness diagnostics are recorded in Appendix A.

The result is evidence of tested scale and useful high-recall operating regions, rather than arbitrary-size scalability. The largest completed search uses 17.78 million vectors on one GPU; there is no multi-GPU experiment, no scaling curve holding data distribution fixed and no construction-peak proof near the card's capacity. Stella's searches use its actual 999-query pool.

7.8 Numerical Robustness, Anomalies and Recall Attribution

The robustness checks qualify the RQ2 exactness claims, the RQ3 risk assessment and the small cross-system margins in RQ4.

7.8.1 Quality resolution and repeated execution

With $k = 10$ and 1,000 queries, one recovered neighbour changes aggregate Recall@10 by 10^{-4}; with 500 held-out queries the resolution is 2×10^{-4}. Stella's 499-query split has a slightly coarser resolution. This discretization is not an uncertainty interval, but it helps interpret the small differences reported above. A fourth-decimal change may represent only one or two result slots, while a small mean loss can still hide several queries with substantially changed answers.

Repeated timing calls must also be separated from repeated index builds. The E1 parser records 369 baseline operating points with a false identifier-determinism flag: 170 RaBitQ, 80 IVF-PQ, 68 CAGRA int8, 50 CAGRA fp32 and one brute-force point. This flag can include result order changes at ties; it does not by itself imply changed

identifier sets or invalid output. No such point is discarded solely for that flag. The raw logs retain first/last recall and set/order diagnostics. JHQ does not populate this same baseline flag, so its absence cannot be counted as a uniform cross-method determinism test. The earlier E1 report's count of 401 uses the union of these 369 repeat-call A flags and 367 B-versus-A mismatch flags; 32 points add only the latter. B-versus-A disagreement is a cross-protocol equality check, not evidence of repeated B calls being nondeterministic.

The five-round throughput summaries retain variability within a fixed build. Minima and maxima are descriptive ranges, not confidence intervals, and they do not account for training variation. Small throughput margins such as the two LUT near ties, or the arXiv–RaBitQ comparison at recall 0.97, consequently receive limited interpretive weight. Larger repeated gains with a matching stage-time reduction provide stronger evidence for an execution improvement than a small frontier displacement alone.

Across each dataset's RaBitQ configurations, the median five-round range divided by median QPS is 3.79%, 3.34%, 2.87% and 5.25% for Vogue, arXiv, OpenAI-1536 and OpenAI-3072. These descriptive spreads are comparable in scale to the small Vogue advantage, rather than confidence intervals for its matched ratio. Raw call traces also contain long calls, including occasional 30–60 ms observations in otherwise faster configurations. As a sensitivity check, recomputing QPS as $10^6/\,\text{median(call latency in ms)}$ for a 1,000-query call reduces the influence of long calls. At recall 0.97, the resulting JHQ/RaBitQ ratios are 1.063, 0.995, 1.118 and 1.100. The three favourable directions and arXiv near parity remain, so the result is not solely a consequence of long calls.

7.8.2 Exactness is attached to a particular stage

Several distinct claims must remain separate. Cartesian factorization preserves the represented score algebraically. Packed loads preserve the logical code values. Complete-key local retention and merging recover the exact primary top-c of the visited candidates. The shared bound preserves that selection target. The FP32 probe primitive passes its synthetic and real-input equivalence checks. None of these establishes exact nearest-neighbour answers in the original vector space.

Lane reductions can alter rounding; primary cutoff ties can change which survivor is refined. Final selection is exact over all retained refined scores with ID-based tie breaking; any remaining error against raw-space neighbours can enter before

Table 7.11: Answers to the four research questions, supported by this chapter.

RQ	Evidence	Answer	Boundary
RQ1	Cold build; resident components; large searches	17.78M vectors on one GPU; 20.44 GiB; 10.40 s train+add	One cold record; no construction peak or multi-GPU scaling
RQ2	Paired LUT, refinement, scan, bound and probe tests	Exact tested primary selection; scheduling 2.727× at batch 8	Ties and numerical paths; full-batch gains much smaller
RQ3	Disjoint query halves; fixed 64/100; oracle; enumeration	Mean QPS wins in 9/12 versus 64, 10/12 versus 100	Calibration overhead; up to 40/64 oracle-relative loss crossings; no drift test
RQ4	Six-method E1 sweeps; matched recall; memory	Three leads and arXiv near parity at 0.97; lower net allocation than fp32 CAGRA	Int8 wins at low recall; endpoint and timing uncertainty

that final selection or through the represented scores. The archived Vogue cross-version discrepancy shows why a pipeline-wide identity claim requires its own check. Similarly, the FP16 routing experiment demonstrates that an almost unchanged mean recall is insufficient evidence of exact probe equivalence. The main correctness conclusions are therefore stated at the stage and input coverage actually tested.

Recall loss can enter at routing, primary screening, residual scoring or final selection. Increasing probes expands routing coverage; increasing α gives more primary survivors access to residual scoring; neither restores information lost by every later approximate stage. The budget sweep directly measures the consequence of changing survivor count while holding the index and probe configuration fixed. It does not decompose every missing neighbour into a unique cause. A complete per-query attribution would require ground-truth coverage checks after routing, after primary retention and after refinement, with an exact final-selection reference on the same candidate set. That full attribution is not established by the experiments used here, and no percentages are assigned to unmeasured sources of loss.

7.8.3 Scope of the research findings

Table 7.11 closes the evidence chain for RQ1–RQ4. The results favour a small set of representation-aware execution modes and workload-specific refinement effort, with independent evidence for correctness, speed and quality.

The claims apply to squared-L2 top-10 search, the recorded query pools and the tested RTX 5090 environment. Cross-device behaviour, changing queries, online updates and open-loop service latency remain untested. These boundaries accompany the results rather than replacing their positive findings.

Chapter 8

Discussion, Limitations and Future Research

TO WRITE: Synthesize supported findings, analyse two relevant negative designs, and state internal and external validity limits.

8.1 Three Cross-Layer Insights

TO WRITE: Develop the supported links between representation and execution, parallelism and repeated work, and execution cost and the value of budget reduction.

8.2 Two Deep Negative-Result Case Studies

TO WRITE: For two case studies, state hypothesis, alternatives, test conditions, observations, and stopping decision. Label isolated GPU, offline, estimated, and untested evidence.

8.3 Threats to Internal and External Validity

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

8.4 Original Testable Research Directions

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

8.5 Conditional Deployment Recommendation

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Chapter 9

Conclusion

TO WRITE: Answer the research questions using only results already established. Do not introduce new measurements or contributions.

9.1 Answers to the Four Research Questions

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

9.2 Closing Principle and Outlook

TO WRITE: Write this section using the verified methods and evidence. State the relevant conditions and connect the argument to its research question.

Bibliography

[1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. *Proc. VLDB Endow.*, 9(4):288–299, 2015. doi: 10.14778/2856318.2856324.

[2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. Mach. Intell.*, 43(5):1666–1677, 2021. doi: 10.1109/TPAMI.2019.2952606.

[3] Oded Green, Robert McColl, and David A. Bader. GPU Merge Path: A GPU merging algorithm. In *Proceedings of the 26th ACM International Conference on Supercomputing*, pages 331–340, 2012. doi: 10.1145/2304576.2304621. URL `https://davidbader.net/publication/2012-gm-ba/2012-gm-ba.pdf`.

[4] Jiaxin Han, Mengxuan Zhang, and Goce Trajcevski. JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow.*, 19(7):1530–1543, 2026. doi: 10.14778/3801059.3801067.

[5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1): 117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[6] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs, 2017. URL `https://arxiv.org/abs/1702.08734`.

[7] NVIDIA. *CUDA C++ Programming Guide, Release 13.0.1*. NVIDIA, 2025. URL `https://docs.nvidia.com/cuda/archive/13.0.1/cuda-c-programming-guide/index.html`. Sections 10.14.1.4 and 18.5.3.3; accessed 27 October 2026.

[8] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In *IEEE ICDE*, 2024. doi: 10.1109/ICDE60146.2024.00323.

[9] Jianyang Shi, Jianyang Gao, Jiaxin Xia, Tamás B. Fehér, and Cheng Long. GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. *Proc. VLDB Endow.*, 19(11):3454–3467, 2026. doi: 10.14778/3836663.3836701.

Appendix A

Reproducibility and Experiment Manifest

A.1 Source and Build Environment

Chapter 7 is an offline analysis of archived measurements in the sibling `JHQ_GPU` repository. The analysis checkout is recorded in `data/ch7/audit.json`; this is distinct from each experimental run's source head. Table A.1 maps component names to implementations, recorded provenance and executable names. A run-head commit identifies source visible to a run, not a cryptographic identity of its binary. The archives do not supply a complete collection of executable SHA-256 hashes or immutable trained-state hashes. Missing identities are left explicit rather than reconstructed from the current checkout.

The principal environment is RTX 5090, driver 595.71.05 and CUDA 13.0; E1 uses cuVS 26.08.01. Release JHQ targets compute_80 PTX, JIT-compiled on the device. Native-library baselines and C++/Python interface boundaries differ. E1 improves comparability by reusing resources, preallocating outputs, using pinned input staging and timing int8 query quantization. JHQ and baseline output buffers still have different host memory types. These details limit interpretations of small margins.

The adaptive call budget uses a warm-up interval containing the first call. In the Python baseline harness that interval also includes initial result conversion and validation before the decision; it is not a pure steady-state latency measurement. The 274 reduced-call points therefore include a RaBitQ point above 507 kQPS. Actual calls and five round arrays remain in the JSONL records. The default exact-selection setting was also inspected at the budget run's historical source head, not merely in the current checkout: `JHQ_EXACT_TOPCK=0` opts into the old lossy primary-retention implementation. Final selection scans all retained refined scores and breaks ties by database ID.

Table A.1: Component and run provenance. "Report" distinguishes a version stated in an experiment report from a raw run-header commit. The exact E1 build state is not inferred from a baseline's metadata.

Component	Recorded source evidence	Executable or harness	Run/state source
E1 final pipeline	Environment logs include `a31c22d`; extensions have their own metadata	`demo_jhq_v69_batch`; `bench_cuvs.py`; `bench_rabitq_sweep`	E1 `RAW_RESULTS/`; shared trained-state cache
Lane refinement (v61)	Raw header `b37efa4`	`demo_jhq_v61_ab`	`warp_refine/raw/run.log`; `arms6_cache`
Grouped scan (v62)	Report: `862543a`; run script `e0d0518`	`demo_jhq_v62_batch`	`pipeline_v62/raw/`; recorded compiler flags
Shared bound (v63)	Implementation and build flags archived; no run-head claim inferred here	`demo_jhq_v63_batch`	`pipeline_campaign/exp2/v63_raw/`
Batch scheduling (v65)	Raw header `d425ab8`	`demo_jhq_v65_batch`	`exp47/raw/run.log`; `arms6_cache`
Radix probe (v69)	Comparative runs and source archived; binary SHA not recorded	`demo_jhq_v65_batch`, `demo_jhq_v69_batch`	`practical_opt/v69_run.sh`; `v69_results/dumps.sha256`
Budget / table sweep	Raw header `d5e1526`	`demo_jhq_v69_budget_arms`; `demo_jhq_v71_gN_batch`, $N \in \{1, 2, 4, 8\}$	`adc_v69/RAW/`, `RAW_LUT/`; `arms6_cache`
Large int8 rerun	Stella degree-64 metadata `ad433a4`	`bench_cuvs.py`	`adc_v69/RAW_BIG/`; isolated processes
Historical budget	Raw header `bf31b41` (v57 family)	`demo_jhq_budget_arms.cu` harness	`paper_adc2026/data/budget_arms.log`

A.2 Data and Trained States

The six database shapes and actual query counts are stated in Chapter 7. The comparisons use the same exported vectors and squared-L2 ground truth per dataset. The budget split uses even query indices for calibration and odd indices for evaluation; Stella has 500/499 queries. Search sweeps load the recorded `arms6_cache` trained states, whereas cold-build records explicitly start without trained state. The chapter's file ledger hashes analysis inputs, not the full underlying vector files or every cache object; it does not certify unrecorded data or binary identities.

The Vogue probe comparison stores matching SHA-256 hashes for the one-group, batch-8 probe/primary dumps, covering all 1,000 queries. Its baseline identity run reports recall 0.9642; the same baseline binary's two subsequent timing runs report

Table A.2: Chapter 7 result-to-figure mapping. Paths below are relative to `JHQ_GPU`; detailed file identities are in the hash ledger.

Figure	Raw source
7.1	`experiments/comp8800_poster_e1/RAW_RESULTS/`; `analyze.py` frontier and interpolation definitions
7.2	`experiments/adc_v69/RAW_LUT/v71_g*.json`
7.3	`paper_adc2026/data/layout6.log`; `experiments/warp_refine/raw/ab/` and `raw/summary.json`
7.4, 7.5	`experiments/adc_v69/RAW/budget_v69.log`; actual 500/499 query counts
7.6	`experiments/pipeline_campaign/exp47/raw/bench/`; `experiments/pipeline_v62/raw/perf/stages_summary.json`
7.7	E1 and `adc_v69/RAW_BIG/` cold-build logs; baseline index records, with int8 dataset quantization
7.8	`adc_v69/RAW_LUT/v71_g2_*_np128.json`; modelled component sizes in the chapter analysis script
7.9	`experiments/adc_v69/RAW_BIG/` corrected top-level records

0.9644. The dump controls a specific routing/screening boundary. It does not store a complete cross-process residual-code, query-rotation or score trace and therefore does not identify the numerical root cause downstream.

Two exclusions affect the scale evidence. Initial large-baseline builds retained an earlier index through a closure; successful isolated reruns supersede those attempts, including Stella CAGRA int8. Separately, Stella's 999-query pool causes the full-batch harness to skip batch 1,000. Top-level corrected large-search records use batch 999; superseded skipped-search files supply no throughput points. Cold-build logs are used for train/add time even where their search list is empty.

A.3 Result-to-Figure Mapping

Table A.2 maps the empirical figures to their measurement families. All Chapter 7 empirical plots and numeric tables are generated by `scripts/build_ch7.py`; `data/ch7/sources.csv` gives consumed input paths relative to `JHQ_GPU` and their hashes. Historical v57 and newer budget comparisons retain separate versions and timing protocols. The memory decomposition combines measured totals with an explicit component model; its unaccounted remainder remains visible.

To regenerate the report, run `make ch7-data` and then `make pdf`. The optional Chinese reading copy uses `make pdf-zh`; its table translator reuses numeric data from the English tables. Regeneration is an offline consistency check, not a new GPU measurement. The original vectors, trained states, matching GPU software

environment and executable build would be needed for an independent benchmark rerun.

Appendix B

Supplementary Results and Correctness Checks

B.1 Complete Parameter Grids

TO WRITE: List method-specific tuning grids, invalid or unavailable settings, and reasons for unattempted configurations.

B.2 Correctness and Numerical Checks

TO WRITE: Include reference comparisons for encoding, LUT factorisation, primary selection, probe selection, and refinement. Separate numerical, algorithmic, and memory-safety evidence.

B.3 Additional Results and Negative Designs

TO WRITE: Include secondary results with evidence status: integrated system, isolated GPU measurement, offline replay, analytical estimate, or untested idea.

Appendix C

Contributions and Tool-Use Record

TO WRITE: Use the form of disclosure required by Canvas. The tables below are a record template, not an attestation that any particular verification was performed. Replace all entries with a truthful account.

C.1 Contribution Record

Work component	Contributor and role	Supporting record
To complete	*To complete*	*To complete*

C.2 Generative-AI Assistance

Tool and date	Purpose and scope	Prompt record and human review
To complete	*To complete*	*To complete*

Appendix D

Template Examples — Draft Only

This appendix demonstrates the available formatting. It is omitted when `\draftthesisfalse` is selected in `config/metadata.tex`.

D.1 Citations and Cross-References

Use `\citet{jhq}` for a narrative citation: Han et al. [4]. Use `\citep{pq,cagra}` for a parenthetical citation [5, 8]. Examples: figure D.1, table D.1, and equation (D.1). The bibliography entries are copied from the existing ADC source; review their metadata before submission.

D.2 A Source-Native Diagram

D.3 Tables

D.4 Equations and Proofs

$$D(\alpha) = \sum_{q \in Q_s} \left[k - \left| R_\alpha(q) \cap R_{\max}(q) \right| \right]. \tag{D.1}$$

The environments `definition`, `lemma`, `proposition`, and `proof` are available. For example:

Definition D.1 (Strict total ranking key)**.** A ranking key specifies how to order unequal scores and how to break ties.

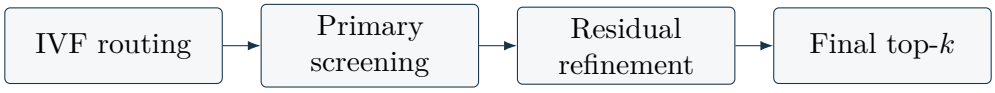


Figure D.1: Formatting example: a schematic query path. Replace or extend it with the verified execution boundaries of the implementation.

Table D.1: Example manifest fields. This table contains no experimental results.

Field	What to record
Source	Immutable commit and build configuration
Index	Trained-state identity and input hashes
Timing	Host-to-host or device-resident boundary
Quality	Metric, ground truth, and held-out split
Evidence	Raw log and analysis-script path

Table D.2: Pseudocode layout example.

Line	Operation
1	Read the input and validate the stated preconditions.
2	Compute the intermediate result.
3	Return the output and specify its semantics.

D.5 Pseudocode and Code

The table below gives a package-light pseudocode layout; replace its contents with the actual method and keep correctness premises in the surrounding text.

Listing D.1: Code-listing format example.

```
// Use only short excerpts that help explain a design.
// Keep full implementations in the reproducibility artifact.
void search();
```